

Portale per il Diabete di tipo 2

Autori

Giacomo Mantovani & Alessia Giacomelli

SOMMARIO

1. Contesto del Sistema

- 1.1 Descrizione del Progetto
 - 1.2 Architettura
 - 1.3 Stack Tecnologico
-

2. Requisiti

- 2.1 Requisiti Funzionali
 - Gestione dell'account e dell'accesso
 - Configurazione dell'account
 - Attività del Paziente
 - Messaggi
 - Attività del Medico
 - Sistema e Amministrazione
 - 2.2 Requisiti Non Funzionali
-

3. Use Case

- 3.1 Attori
- 3.2 Use Case
 - Comuni
 - Paziente
 - Medico
 - Amministratore
 - Sistema
- 3.3 Casi Comuni
 - UC-01: Login
 - UC-02: Cambio Password
 - UC-03: Chiusura della Sessione

- **3.4 Paziente**
 - UC-10: Completamento del Profilo Clinico
 - UC-11: Scelta del Medico
 - UC-12: Registrazione di una Misurazione
 - UC-13: Andamento Glicemico
 - UC-14: Registrazione Assunzione di un Farmaco
 - UC-15: Terapie Attive
 - UC-16: Segnalazione di un Problema
 - UC-17: Visualizzazione Notifiche
 - UC-18: Chat con il Medico
 - **3.5 Medico**
 - UC-20: Completamento del Profilo del Medico
 - UC-21: Sintesi Lavorativa
 - UC-22: Visualizzazione Pazienti
 - UC-23: Dettagli Paziente
 - UC-24: Aggiornamento Cartella Clinica del Paziente
 - UC-25: Prescrizione di una Terapia
 - UC-26: Modifica o Disattivazione di una Prescrizione
 - UC-27: Prendere in carico una Segnalazione
 - UC-28: Chat con un Paziente
 - **3.6 Amministratore**
 - UC-30: Registrazione di un Utente
 - UC-31: Visualizzazione Utenti
 - **3.7 Sistema**
 - UC-40: Generazione Avviso Glicemico
 - UC-41: Generazione Avviso Assunzione
 - UC-42: Rilevazione Mancata Aderenza alla Terapia
-

4. Architettura

- **4.1 Livelli**
 - Presentazione
 - Controllo
 - Servizio
 - Persistenza
 - **4.2 Richieste HTTP**
-

5. Database

- 5.1 Persone
 - 5.2 Fattori di Rischio e Patologie
 - 5.3 Chat e Alert
 - 5.4 Mappatura
-

6. Progettazione

- 6.1 Dominio
 - Persone
 - Dati Clinici
 - Terapie
 - Comunicazione
 - 6.2 Persistenza
 - 6.3 Controllo
 - 6.4 Utilità e Servizio
-

7. Scenari di Interazione

- 7.1 Accesso e Cambio Password
 - 7.2 Registrazione di una Misurazione
 - 7.3 Registrazione di un'Assunzione
 - 7.4 Medico Apre i Dettagli di un Paziente
 - 7.5 Medico Prescrive una Terapia
 - 7.6 Presa a carico di una Segnalazione
 - 7.7 Invio di un Messaggio
 - 7.8 Controllo Terapie
-

8. Package

- **8.1 Package model**
 - User , Patient e Doctor
 - Role
 - MealMoment
 - AlertSeverity
 - GlycaemiaLevel
 - MissedIntake
 - **8.2 Package persistence**
 - MySqlConnectionManager
 - JdbcDAOFactory
 - **8.3 Package web**
 - AbstractPortalServlet
 - AbstractPatientServlet e AbstractDoctorServlet
 - LoginServlet
 - PatientServlet
 - DoctorServlet
 - AdministrationServlet
 - Servlet di Servizio
 - Pages
 - Html
 - Flash e FlashMessage
 - ApplicationContextListener
 - **8.4 Package service**
 - AdherenceMonitor
-

9. Test

- **9.1 Obiettivo e Metodologia**
 - **Ambiente di Test**
 - **Come sono stati Eseguiti i Test**
 - **Riproducibilità: Script e Comandi usati**
 - **9.2 Leggenda**
 - **9.3 - Tabella dei Test Eseguiti**
 - **Integrità del Codice**
 - **Autenticazione e Sessione**
 - **Area Amministrazione**
 - **Primo Accesso e Area Medico**
 - **Primo Accesso e Area Paziente**
 - **Misurazioni Glicemiche**
 - **Terapie e Assunzioni farmaci**
 - **Segnalazione Problemi/Sintomi**
 - **Segnalazioni e avvisi Alert**
 - **Messaggistica Paziente-Medico**
 - **Controllo Aderenza alla Terapia**
 - **Sicurezza Applicativa**
-

1 - Contesto del Sistema

1.1 - Descrizione del Progetto

Si deve creare un sistema di telemedicina di un servizio clinico per la gestione di pazienti diabetici.

Nel sistema ci dovranno essere tre attori principali: paziente, medico e amministratore.

Il paziente dovrà interagire con il sistema per monitorare le misurazioni della glicemia e per gestire la propria terapia.

Il medico interagisce con il sistema per gestire i propri pazienti, ai quali dovrà affidare terapie, prescrizioni e visualizzare gli andamenti delle misurazioni della glicemia.

L'amministratore dovrà interagire con il sistema per la gestione degli utenti nel sistema, infatti l'amministratore dovrà registrare i nuovi utenti (pazienti o medici o amministratori).

Il sistema poi dovrà verificare che il paziente mantenga la terapia attiva, altrimenti dovrà segnalare al medico di riferimento il fatto. Oltre al verificare di altri alert, come per problemi nella misurazione della glicemia. Inoltre il sistema deve mantenere un sistema di mail/chat tra medico di riferimento e paziente.

1.2 - Architettura

Per il sistema si è scelta una architettura client-server, poiché il servizio offerto deve essere accessibile da qualunque postazione senza richiedere nessuna installazione, il client è il browser, quindi non esiste nessuna versione per sistema operativo da mantenere e distribuire.

La ragione principale riguarda però la gestione dei ruoli e l'accesso ai dati. Con applicazioni installate su singole postazioni, ciascuna avrebbe dovuto contenere le credenziali d'accesso al database, portando a possedere l'intera base di dati e il controllo sui ruoli.

Inoltre il sistema deve verificare in background alcuni dati, in quanto l'applicazione dovrebbe essere sempre in esecuzione sulla posizione, questa feature non potrebbe essere garantita.

1.3 - Stack Tecnologico

Livello	Tecnologia	Versione	Ruolo	Perché
Linguaggio	Java	17	Tutta la logica applicativa	Versione LTS; source e target fissati nel pom.xml
Piattaforma	Jakarta EE Servlet, JSP	11 (Servlet 6.0)	Gestione di richieste, sessioni, generazione delle pagine	API standard, senza framework aggiuntivi: il ciclo della richiesta resta visibile
Contenitore	Apache Tomcat	10 o superiore	Esecuzione dell'applicazione	Contenitore servlet leggero; dalla versione 10 usa lo spazio dei nomi jakarta.*
Base dati	MySQL	8	Persistenza dei dati clinici	Supporto completo alle transazioni e all'integrità referenziale
Accesso ai dati	JDBC mysql-connector-j	8.3.0	Comunicazione con il database	SQL esplicito e visibile, senza livelli di astrazione intermedi
Sicurezza	jBCrypt	0.4	Hash delle password (costo 12)	Algoritmo per criptare le password degli utenti
Utilità	Apache Commons Text	1.11.0	Escape dell'HTML, generazione di password casuali	Difesa contro l'XSS; generazione crittograficamente sicura
Presentazione	HTML5, CSS3		Interfaccia	Fogli di stile scritti a mano, senza framework CSS
Grafici	Chart.js	via CDN	Andamento glicemico	Disegna il grafico tramite i dati recapitati tramite la sessione
Costruzione	Apache Maven	3.8+	Compilazione e pacchettizzazione	Artefatto unico: un archivio WAR

2 - Requisiti

2.1 - Requisiti Funzionali

Gestione dell'account e dell'accesso

Per la gestione dell'accesso e gestione dell'account si sono identificati i seguenti requisiti:

Id	Requisito	Attore	Priorità
RF-01	L'amministratore registra un utente indicando nome, cognome, mail e ruolo. Il sistema genera una password	Amministratore	Massima
RF-02	L'utente accede tramite mail e password	Tutti	Massima
RF-03	Al primo accesso il sistema obbliga a cambiare la password.	Tutti	Massima
RF-04	Password deve essere lunga almeno 8 caratteri e deve essere confermata dall'utente	Tutti	Massima
RF-05	L'utente viene demandato all'area del proprio ruolo	Tutti	Massima
RF-06	L'utente può fare il logout in qualsiasi momento	Tutti	Massima
RF-07	La sessione scade dopo una inattività di 30 minuti	Sistema	Media

Configurazione dell'account

Si sono poi identificati i requisiti per la configurazione iniziale del profilo dell'utente del sistema:

Id	Requisito	Attore	Priorità
RF-08	Al primo accesso si inseriscono i dati anagrafici	Paziente Medico	Massima
RF-09	Vengono dichiarati i fattori di rischio che riguardano l'utente	Paziente	Massima
RF-10	Vengono dichiarate le patologie pregresse	Paziente	Media
RF-11	Il paziente sceglie il proprio medico di riferimento tra i medici registrati nel portale	Paziente	Massima

Attività del Paziente

Si sono identificati i requisiti del **Paziente**, il quale dovrà utilizzare il portale ogni giorno, per più volte. Si è identificata la routine quotidiana del paziente:

Id	Requisito	Attore	Priorità
RF-12	Registrazione di una misurazione della glicemia indicando il valore e il momento della registrazione rispetto ai pasti	Paziente	Massima
RF-13	Il sistema classifica il valore, se è fuori da un certo intervallo si genera un avviso al medico con un livello di gravità	Sistema	Massima
RF-14	Il paziente vede l'andamento glicemico tramite un grafico, scegliendo quanti punti vedere	Paziente	Media
RF-15	Il paziente vede lo storico delle misurazioni	Paziente	Media
RF-16	Il paziente registra l'assunzione di un farmaco, prescritto, indicando la quantità presa	Paziente	Massima
RF-17	Il sistema verifica l'assunzione del farmaco sulla base della prescrizione, se l'assunzione non rispetta la prescrizione il sistema manda un avviso al medico	Sistema	Massima
RF-18	Il paziente vede le proprie terapie attive, con il relativo dosaggio del farmaco e la frequenza.	Paziente	Massima
RF-19	Il paziente segnala un problema, indicando uno o più sintomi. Inoltre indicando se il sintomo riguarda una terapia o una patologia pregressa	Paziente	Massima
RF-20	Il paziente consulta le proprie notifiche, in un pannello dedicato, che mostra se sono state prese, oppure no, in carico dal medico	Paziente	Media

☰ Soglie RF-17

Il sistema valuta se il dosaggio è corretto o meno tramite percentuali. Se l'assunzione è entro il **10%** della dose prescritta, il sistema considera l'assunzione regolare e non crea nessun avviso. Se il paziente effettua un'assunzione che supera il **10%** il sistema crea un avviso di gravità di **WARNING**. Se invece il paziente assume il **doppio** oppure **metà** la gravità dell'avviso sarà **CRITICAL**.

Messaggi

Si identificano i requisiti della chat tra medico-paziente:

Id	Requisito	Attore	Priorità
RF-21	Il paziente scrive al proprio medico di riferimento in modo diretto	Paziente	Media
RF-22	Il medico sceglie il paziente assistito e può scrivergli	Medico	Media
RF-23	Le chat mostrano gli ultimi messaggi in ordine cronologico, mostrando visivamente il mittente	Tutti	Media

Attività del Medico

Si identificano le attività che un medico può effettuare su un proprio paziente assistito:

Id	Requisito	Attore	Priorità
RF-24	Nella pagina home vengono mostrati i numeri di sintesi (assistiti-terapie attive- segnalazioni)	Medico	Massima
RF-25	Medico consulta l'elenco dei propri pazienti a carico	Medico	Massima
RF-26	Ogni dettaglio dei pazienti è diviso in due colonne: a sinistra la funzionalità e a destra il grafico/form riguardante	Medico	Massima
RF-27	IL grafico del dettaglio riguardante la glicemia si può filtrare per ultima settimana o ultimo mese	Medico	Media
RF-28	Il medico consulta e aggiorna la cartella clinica di un paziente	Medico	Massima
RF-29	Il medico prescrive una terapia a un paziente, indicando tutti i dati relativi ad essa	Medico	Massima
RF-30	Il medico consulta le segnalazioni aperte dai pazienti e prenderle a carico	Medico	Massima

Sistema e Amministrazione

Id	Requisito	Attore	Priorità
RF-31	Il sistema, in modo periodico, deve rilevare che i pazienti, con terapie in corso, se non registrano assunzioni da un limite di tempo prestabilito. In tal caso viene generato un avviso al medico di riferimento	Sistema	Massima
RF-32	L'amministratore consulta gli elenchi di pazienti e medici e i conteggi degli utenti per ruolo	Amministrazione	Massima

2.2 - Requisiti Non Funzionali

Si sono identificati diversi requisiti non funzionali sulle basi di: sicurezza, prestazioni e manutenibilità.

Requisito	Verifica	Categoria
Le password non devono essere memorizzate in chiaro, devono essere salvate come hash tramite BCrypt con costo 12	<code>UTENTE.password</code> deve iniziare con <code>\$2a\$12\$</code>	Sicurezza
La password temporanea è generata tramite <code>SecureRandom</code> e non tramite <code>Math.random</code>	<code>JdbcUserDAO.register</code>	Sicurezza
Dopo l'autenticazione la sessione precedente viene invalidata e ne viene creata una nuova con un nuovo identificativo	cookie <code>JSESSIONID</code>	Sicurezza
Il cookie di sessione è impostato a <code>HttpOnly</code> . Il flag <code>Secure</code> è predisposto e va attivato in HTTPS	<code>web.xml</code> <code><cookie-config></code>	Sicurezza
Ogni valore di una variabile stampato in una pagina passa per l'escape HTML	si verifica che appaia come testo	Sicurezza
Le query in SQL usano <code>PreparedStatement</code> con i parametri	nelle query si usa l'escape <code>?</code> dove si deve immettere il dato	Sicurezza
La pagina di errore non mostra mai stack trace o nomi di classi all'utente	pagina html di errore	Sicurezza
Le pagine JSP non interrogano il database, ricevono gli attributi e li processano per la pagina grafica	Reindirizzamento verso le pagine JSP tramite <code>doGet</code> e <code>doPost</code>	Prestazione
Un solo punto decide quale implementazione dei DAO viene usata, nessuna classe istanza un <code>Jdbc</code> direttamente	Si deve trovare solo la factory	Manutenzione
Il compito periodico non può essere ucciso da un'eccezione e viene spento in modo ordinato allo spegnimento del contesto	<code>AdherenceMonitor.run</code> e <code>AppContextListener</code>	Manutenzione

3 - Use Case

3.1 - Attori

Per definire i diversi use case del sistema, bisogna prima identificare gli attori e il loro ruolo. Nell'intero sistema si identificano quattro attori:

1. Paziente
2. Medico
3. Amministratore
4. Sistema

PAZIENTE	
Definizione	Persona affetta da diabete
Obiettivi	Registrazione e visualizzazione dei propri dati clinici, misurazioni della glicemia e assunzioni dei farmaci, visualizzazione di avvisi e la loro presa a carico e comunicazione con il proprio medico
Frequenza d'uso	Più volte al giorno
Competenze presupposte	Uso di un browser per l'accesso al portale
Vincolo distintivo	Ha esattamente un medico di riferimento

MEDICO	
Definizione	Persona professionista che segue uno o più pazienti del portale
Obiettivi	Consulente verso i pazienti con terapie e prescrizioni sulla base del paziente. Visualizzare e prendere a carico avvisi arrivati dai propri pazienti.
Frequenza d'uso	Quotidiana/sessioni
Competenze presupposte	Uso di un browser per l'accesso al portale e competenze cliniche per interpretare i dati
Vincolo distintivo	Ha molti pazienti e accede esclusivamente ai propri

AMMINISTRATORE	
Definizione	Persona che gestisce le anagrafiche degli utenti del portale
Obiettivi	Creare gli account di pazienti, medici e altri amministratori. Conoscere la "popolazione" degli utenti
Frequenza d'uso	Occasionale
Competenze presupposte	Uso di un browser per l'accesso al portale
Vincolo distintivo	Non accede a nessun dato

SISTEMA	
Definizione	Portale stesso quando agisce senza che nessuno gliel'abbia chiesto
Obiettivi	Rendere visibile al medico avvisi riguardanti i propri pazienti
Frequenza d'uso	A ogni registrazione di un dato e ogni giorno per il controllo della continuità delle terapie
Competenze presupposte	-
Vincolo distintivo	Attore temporale

3.2 - Use Case

Si identificano i seguenti use case in base agli attori del sistema, si enumerano gli use case in base all'attore che lo utilizza.

Comuni

Tramite il codice iniziale 0 si identificano gli use case comuni a tutti gli attori umani:

UseCase	Nome	Attore	Requisiti
UC-01	Accedere al portale	Paziente, Medico, Amministratore	RF-02, RF-05
UC-02	Cambiare la password iniziale	Paziente, Medico, Amministratore	RF-05, RF-04
UC-03	Chiudere la sessione	Paziente, Medico, Amministratore	RF-06

Paziente

Tramite il codice iniziale **1** si identificano gli use case relativi al **Paziente** :

UseCase	Nome	Attore	Requisiti
UC-10	Completare il profilo clinico	Paziente	RF-08 RF-09
UC-11	Scegliere il medico di riferimento e dichiarare le patologie	Paziente	RF-10 RF-11
UC-12	Registrazione di una misurazione della glicemia	Paziente	RF-12
UC-13	Consultare l'andamento glicemico	Paziente	RF-14 RF-15
UC-14	Registrare l'assunzione di un farmaco	Paziente	RF-16
UC-15	Consultare le terapie attive	Paziente	RF-18
UC-16	Segnalare un problema	Paziente	RF-19
UC-17	Consultare le notifiche	Paziente	RF-20
UC-18	Scrivere al proprio medico	Paziente	RF-21 RF-23

Info

Gli use case **UC-10** e **UC-11** sono due use case distinti, ma **UC-11** è raggiungibile solo dopo il primo. Nel sistema questo ordine è imposto a ogni richiesta

Medico

Tramite il codice iniziale 2 si identificano gli use case relativi al Medico :

UseCase	Nome	Attore	Requisiti
UC-20	Completare il profilo del medico	Medico	RF-08
UC-21	Consultare la sintesi degli assistiti	Medico	RF-24
UC-22	Consultare l'elenco degli assistiti	Medico	RF-25
UC-23	Consultare un dettaglio di un proprio paziente	Medico	RF-26 RF-27
UC-24	Aggiornare la cartella clinica di un proprio paziente	Medico	RF-28
UC-25	Prescrivere una terapia	Medico	RF-29
UC-26	Modificare o disattivare una prescrizione	Medico	RF-29
UC-27	Prendere in carico una segnalazione	Medico	RF-30
UC-28	Scrivere a un proprio paziente	Medico	RF-22 RF-23

Amministratore

Tramite il codice iniziale 3 si identificano gli use case relativi all' Amministratore :

UseCase	Nome	Attore	Requisiti
UC-30	Registrare un nuovo utente	Amministratore	RF-01
UC-31	Visualizzare gli utenti in base al loro ruolo	Amministratore	RF-32

Sistema

Tramite il codice iniziale 4 si identificano gli use case relativi al Sistema :

UseCase	Nome	Innesco	Requisiti
UC-40	Generare un avviso dal valore glicemico fuori norma	Incluso in UC-12	RF-13
UC-41	Generare un avviso da una dose non conforme	Incluso in UC-14	RF-17
UC-42	Rilevare la mancata aderenza alla terapia	Scadenza intervallo temporale	RF-31

3.3 - Casi comuni

UC-01: Login

UC-01	
Attore	Paziente, Medico, Amministratore
Requisiti	RF-02, RF-05
Precondizioni	L'utente esiste, creato con UC-30
Postcondizioni	Esiste una nuova sessione contenente l'utente. L'utente si trova sulla pagina del proprio ruolo o del cambio password

Scenario Principale

1. L'utente apre la pagina di login e inserisce le proprie credenziali
2. Il sistema cerca l'utente nel database tramite il suo `username` e confronta la password inserita dall'utente con quella salvata
3. Il sistema invalida la sessione corrente e ne crea una nuova
4. Il sistema registra l'utente nella sessione
5. Il sistema instrada l'utente all'area corrispondente al ruolo dell'utente

Scenari Alternativi

Condizione	Comportamento
Utente inesistente o credenziali errate	Ritorno alla pagina di login con il messaggio "credenziali non valide"
Un campo vuoto	Stampa dello stesso messaggio, ma senza interrogare il db
Primo accesso dell'utente	L'utente viene instradato verso UC-02 invece di raggiungere la sua area dedicata
Ruolo dell'utente non è riconosciuto	La sessione viene invalidata e si demanda alla pagina di errore

UC-02: Cambio Password

UC-02	
Attore	Paziente, Medico, Amministratore
Requisiti	RF-05, RF-04
Precondizioni	Credenziali corrette, account impostato come nuovo nel db
Postcondizioni	Password modificata, l'utente non risulta più come nuovo e viene instradato all'area del proprio ruolo oppure nell'aggiunta delle anagrafiche

Scenario Principale

1. Il sistema mostra il modulo di cambio password
2. L'utente inserisce la nuova password e la conferma ripetendola. Password che deve avere almeno 8 caratteri e i due inserimenti devono coincidere
3. Il sistema calcola l'hash della nuova password e la memorizza
4. Il sistema rimuove il flag di account nuovo
5. Il sistema instrada verso l'area del proprio ruolo oppure nell'aggiunta delle anagrafiche

Scenari Alternativi

Condizione	Comportamento
Password più corta di 8 caratteri o le copie non coincidono	Si mostra all'utente di dover inserire una password conforme e che coincidono
La memorizzazione fallisce	Visualizzazione di un messaggio di errore
La rimozione del flag nuovo fallisce	L'utente all'ingresso successivo rivede il modulo di cambio password

Obbligatorietà del modulo

Una volta che l'utente entra nel modulo non può navigare in altre pagine, se non in quella di login, finché non imposta una nuova password.

UC-03: Chiusura della Sessione

UC-03	
Attore	Paziente, Medico, Amministratore
Requisiti	RF-06
Precondizioni	Esiste una sessione autenticata
Postcondizioni	La sessione viene distrutta rendendo tutte le pagine irraggiungibili senza effettuare un nuovo login

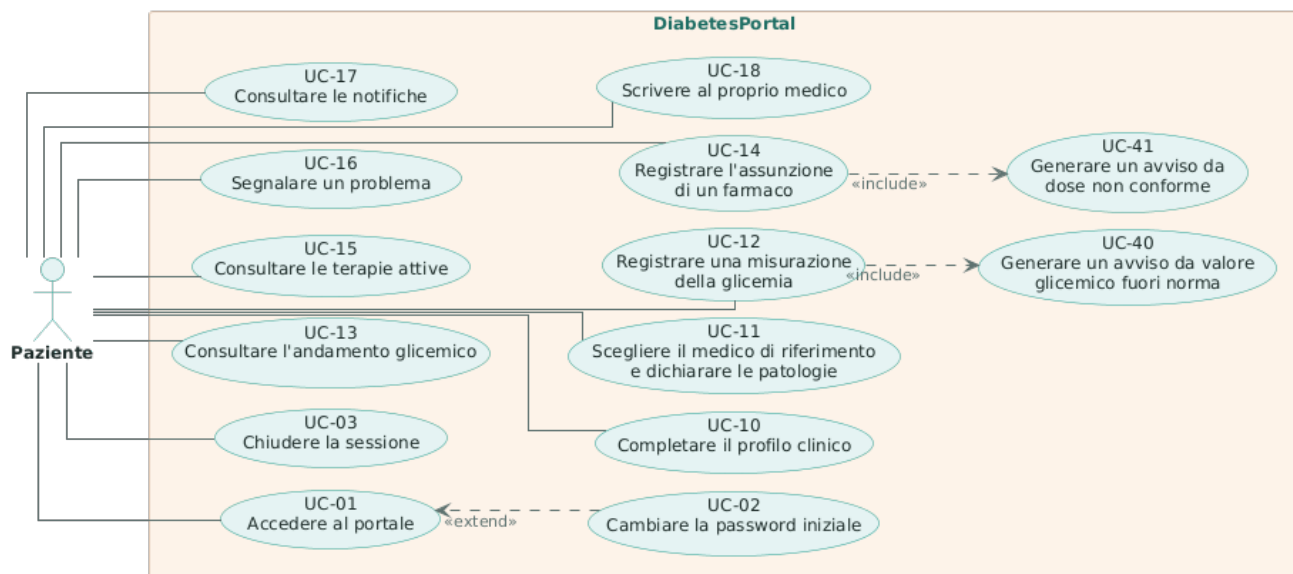
Scenario Principale

1. L'utente richiede la chiusura della sessione dalla sua area
2. Il sistema distrugge la sessione e tutto il suo contenuto
3. Il sistema instrada l'utente alla pagina di login

Scenari Alternativi

Condizione	Comportamento
La sessione è già scaduta	Si raggiunge la pagina di login

3.4 - Paziente



Il paziente è l'attore che genera la maggior parte dei dati del sistema, proprio per questo la maggior parte dei suoi use case hanno la stessa preconditione, tutti tranne quelli di configurazione.

UC-10: Completamento del Profilo Clinico

UC-10	
Attore	Paziente
Requisiti	RF-08, RF-09
Precondizioni	Paziente autenticato e profilo incompleto
Postcondizioni	Vengono creati i dati anagrafici e la cartella clinica del paziente

Scenario Principale

1. Il paziente autenticato raggiunge una qualsiasi pagina della propria area e il sistema rileva che il profilo è incompleto e lo demanda al modulo dell'inserimento dati.
2. Il paziente inserisce data di nascita e sesso
3. Il paziente contrassegna quali fattori di rischio lo riguardano
4. Il sistema registra l'anagrafica e la cartella clinica
5. Il sistema instrada il paziente al passo successivo della configurazione UC-11

Scenari Alternativi

Condizione	Comportamento
Data di nascita assente o non valida	Il modulo torna con il motivo di non accettazione
Il paziente ricarica la pagina e reinvia il modulo	I valori verranno poi aggiornati
Registrazione fallisce	Visualizzato un messaggio di errore e il profilo resta incompleto

UC-11: Scelta del Medico

UC-11	
Attore	Paziente
Requisiti	RF-10, RF-11
Precondizioni	UC-10 concluso e nessun medico di riferimento assegnato al paziente
Postcondizioni	Il paziente ha un medico di riferimento e vengono registrate le patologie pregresse

Scenario Principale

1. Il sistema mostra l'elenco dei medici registrati e l'elenco delle patologie
2. Il paziente sceglie un medico e contrassegna le patologie pregresse che lo riguardano
3. Il sistema associa il medico al paziente e sostituisce l'elenco delle patologie
4. Il paziente viene instradato dal sistema alla propria home nell'area paziente rendendo il portale operativo per lui

Scenario Alternativo

Condizione	Comportamento
Nessun medico registrato nel sistema	l'elenco è vuoto e il paziente non può proseguire, problema risolvibile con UC-30

✓ Portale Operativo

Finché questo use case non viene concluso interamente il paziente non può avere accesso al portale, essendo che registrando dati essi non raggiungerebbero nessuno.

UC-12: Registrazione di una Misurazione

UC-12	
Attore primario	Paziente
Attore secondario	Sistema UC-40 incluso
Requisiti	RF-12
Precondizioni	Profilo completo
Postcondizioni	Viene registrata la misurazione della glicemia, generazione stato glicemico

Scenario Principale

1. Il paziente apre il modulo della misurazione
2. Il paziente inserisce il valore glicemico e il momento rispetto ai pasti
3. Il sistema verifica lo stato glicemico
4. Il sistema registra la misurazione
5. Il sistema classifica il valore glicemico e se necessario genera l'avviso al medico UC-40
6. Il paziente riceve conferma

Scenari Alternativi

Condizione	Comportamento
Valore non numerico o fuori dai limiti di plausibilità	Visualizzato un messaggio di errore
Registrazione fallisce	Visualizzato un messaggio di errore

UC-13: Andamento Glicemico

UC-13	
Attore primario	Paziente
Requisiti	RF-14, RF-15
Precondizioni	Profilo completo
Postcondizioni	-

Scenario Principale

1. Il paziente arriva alla propria pagina home oppure lo storico delle misurazioni
2. Il sistema carica gli ultimi valori registrati e li mostra tramite grafico e/o come elenco

Scenari Alternativi

Condizione	Comportamento
Nessuna misurazione salvata nel db	Visualizzato un messaggio di invito a registrare la prima misurazione glicemica

UC-14: Registrazione Assunzione di un Farmaco

UC-14	
Attore primario	Paziente
Attore secondario	Sistema UC-41 incluso
Requisiti	RF-16
Precondizioni	Profilo completo
Postcondizioni	L'assunzione viene registrata, se la dose non è conforme viene creato un avviso destinato al medico e visualizzato

Scenario Principale

1. Il paziente apre il modulo e vede le proprie terapie attive con la dose prescritta
2. il paziente sceglie la terapia e dichiara la quantità di farmaco assunta
3. Il sistema registra l'assunzione
4. Il sistema confronta la quantità inserita con quella prescritta, se lo scostamento supera la tolleranza genera un avviso UC-41
5. Il paziente riceve conferma

Scenari Alternativi

Condizione	Comportamento
Quantità negativa, non numerica oppure irragionevole	Visualizzato un messaggio di errore
Scostamento entro il 10% della dose prescritta	Nessun avviso creato

UC-15: Terapie Attive

UC-15	
Attore primario	Paziente
Requisiti	RF-18
Precondizioni	Profilo completo
Postcondizioni	-

Scenario Principale

1. Il paziente apre l'elenco delle proprie terapie
2. Il sistema mostra, per ciascuna terapia attiva, farmaco, quantità d'assunzione, frequenza giornaliera ed eventuali indicazioni del medico

Scenari Alternativi

Condizione	Comportamento
Nessuna terapia attiva	Visualizzato un messaggio di elenco vuoto
Terapia disattivata dal medico	Non viene mostrata al paziente

UC-16: Segnalazione di un Problema

UC-16	
Attore primario	Paziente
Requisiti	RF-19
Precondizioni	Profilo completo
Postcondizioni	Il problema viene registrato con tutti i sintomi indicati

Scenario Principale

1. Il paziente apre il modulo e vede l'elenco dei sintomi, le proprie terapie e le proprie patologie
2. Il paziente sceglie uno o più sintomi e eventualmente scrive una descrizione
3. Il paziente collega il problema a una terapia o patologia
4. Il sistema registra il problema e i sintomi associati

Scenari Alternativi

Condizione	Comportamento
Nessuna sintomo scelto	Visualizzato un messaggio di errore
Descrizione più lunga del limite ammesso	Troncata al limite

UC-17: Visualizzazione Notifiche

UC-17	
Attore primario	Paziente
Requisiti	RF-20
Precondizioni	Profilo completo
Postcondizioni	-

Scenario Principale

1. Il paziente apre il pannello delle notifiche
2. Il sistema mostra gli avvisi che lo riguardano, con gli avvisi aperti in cima alla lista e quelli già processati dal medico di seguito
3. Il paziente vede il tipo, la gravità, il testo, la data e lo stato di ogni avviso

Scenari Alternativi

Condizione	Comportamento
Nessun avviso	Visualizzato un messaggio di elenco vuoto

UC-18: Chat con il Medico

UC-18	
Attore primario	Paziente
Requisiti	RF-21, RF-23
Precondizioni	Profilo completo
Postcondizioni	Il messaggio del paziente è registrato nella conversazione

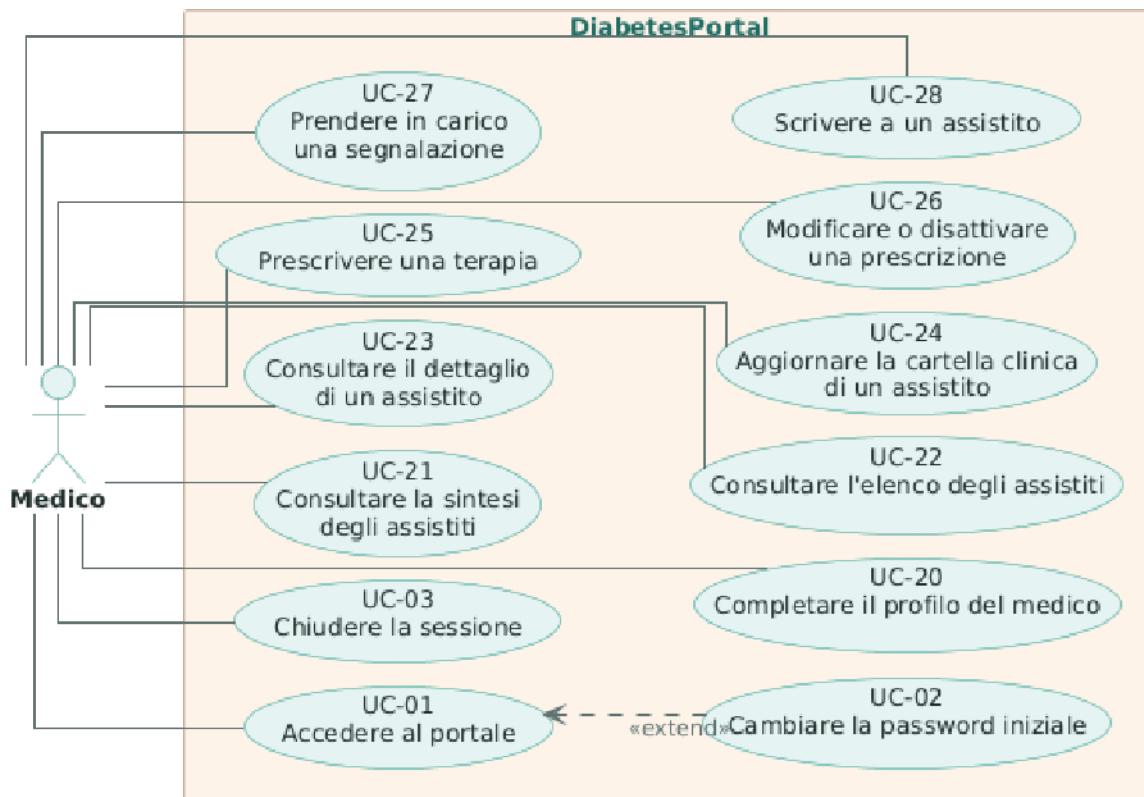
Scenario Principale

1. Il paziente apre i messaggi verso il proprio medico di riferimento
2. Il sistema carica gli ultimi messaggi della conversazione
3. Il paziente scrive un nuovo messaggio e lo invia
4. Il sistema registra il messaggio attribuendolo al paziente e ricarica la conversazione

Scenari Alternativi

Condizione	Comportamento
Testo vuoto o composto da soli spazi	Messaggio rifiutato
Testo più lungo del limite	Troncata al limite
La registrazione fallisce	Visualizzato un messaggio di errore

3.5 - Medico



Il medico è l'attore che interpreta i dati del paziente, creando di conseguenza terapie con assunzioni di un farmaco.

Precondizioni

Gli use case del medico che riguardano un paziente portano la stessa precondizione, ovvero che il paziente è in cura presso il medico

UC-20: Completamento del Profilo del Medico

UC-20	
Attore primario	Medico
Requisiti	RF-08
Precondizioni	Medico autenticato e dati anagrafici assenti
Postcondizioni	Registrazione dei dati anagrafici

Scenario Principale

1. Il Medico a cede e il sistema rileva che il suo profilo è incompleto
2. Il sistema lo porta al modulo dei dati anagrafici
3. Il medico inserisce la propria data di nascita e sesso
4. Il sistema registra i dati e lo demanda alla propria pagina home

Scenari Alternativi

Condizione	Comportamento
Dati assenti o non validi	Visualizzato un messaggio di errore

UC-21: Sintesi Lavorativa

UC-21	
Attore primario	Medico
Requisiti	RF-24
Precondizioni	Profilo del medico completo
Postcondizioni	-

Scenario Principale

1. Il Medico apre la propria pagina home
2. Il sistema calcola tre valori di sintesi: numero di pazienti, terapie attive prescritte e segnalazioni aperte
3. Il sistema carica le segnalazioni più recenti
4. Il medico vede i valori di sintesi e l'elenco delle ultime segnalazioni

Scenari Alternativi

Condizione	Comportamento
Nessun paziente in carico	I valori sono a zero e viene visualizzato un messaggio esplicito
Nessuna segnalazione aperta	Visualizzato un messaggio di elenco vuoto

UC-22: Visualizzazione Pazienti

UC-22	
Attore primario	Medico
Requisiti	RF-25
Precondizioni	Profilo del medico completo
Postcondizioni	-

Scenario Principale

1. Il Medico apre l'elenco dei propri pazienti
2. Il sistema mostra i pazienti in cura presso il medico in ordine alfabetico
3. Da ciascuna voce dell'elenco il medico raggiunge il dettaglio del paziente UC-23

Scenari Alternativi

Condizione	Comportamento
Nessun paziente	Visualizzato un messaggio di elenco vuoto

UC-23: Dettagli Paziente

UC-23	
Attore primario	Medico
Requisiti	RF-26, RF-27
Precondizioni	Profilo del medico completo, paziente in cura presso il medico
Postcondizioni	-

Scenario Principale

1. Il Medico sceglie un paziente dall'elenco
2. Il sistema verifica il legame di cura e carica i dati richiesti
3. Il sistema presenta la pagina divisa in due: a sinistra i comandi delle funzioni cliniche e a destra il grafico/form del dettaglio
4. Da ciascun comando il medico raggiunge: cartella clinica, misurazioni, nuova terapia o prescrizioni

Scenari Alternativi

Condizione	Comportamento
Il paziente non è in cura presso il medico	Il medico viene riportato all'elenco con un messaggio di errore
Dettagli non presenti	Visualizzato un messaggio

UC-24: Aggiornamento Cartella Clinica del Paziente

UC-24	
Attore primario	Medico
Requisiti	RF-28
Precondizioni	Profilo del medico completo, paziente in cura presso il medico
Postcondizioni	Aggiornamento cartella clinica del paziente

Scenario Principale

1. Il Medico apre la cartella clinica del paziente
2. Il sistema mostra i fattori di rischio e le patologie con lo stato attuale
3. Il medico effettua una modifica
4. Il sistema registra i fattori di rischio e sostituisce l'elenco delle patologie

Scenari Alternativi

Condizione	Comportamento
La cartella clinica non esiste ancora	Viene visualizzata la lista dei fattori di rischio e patologie vuoti

UC-25: Prescrizione di una Terapia

UC-25	
Attore primario	Medico
Requisiti	RF-29
Precondizioni	Profilo del medico completo, paziente in cura presso il medico, catalogo dei farmaci non è vuoto
Postcondizioni	Registrazione di una nuova terapia attiva con data d'inizio corrente

Scenario Principale

1. Il Medico apre la funzione nuova terapia per il paziente
2. Il sistema mostra il catalogo dei farmaci prescrivibili
3. Il Medico sceglie il farmaco e indica la quantità da assumere, la frequenza giornaliera ed eventuali indicazioni
4. Il sistema registra la terapia e la imposta come attiva, con inizio la data odierna
5. Il Medico viene demandato all'elenco delle prescrizioni con la nuova voce presente

Scenari Alternativi

Condizione	Comportamento
Elenco dei farmaci vuoto	Nessun farmaco selezionabile, c'è un problema con il database
Quantità del farmaco negativa, non numerica o un valore non finito	Visualizzato un messaggio di rifiuto
Frequenza giornaliera fuori dall'intervallo 1-24	Rifiutato l'inserimento e visualizzato un messaggio
Indicazioni più lunghe del limite ammesso	Troncate

UC-26: Modifica o Disattivazione di una Prescrizione

UC-26	
Attore primario	Medico
Requisiti	RF-29
Precondizioni	Profilo del medico completo, la terapia appartiene a un paziente in cura presso il medico
Postcondizioni	La terapia ha i nuovi valori, oppure risulta sospesa con data di fine

Scenario Principale

1. Il Medico apre l'elenco delle prescrizioni del paziente
2. Il Medico modifica quantità, frequenza o indicazioni di una terapia oppure ne richiede la disattivazione
3. Il sistema esegue l'operazione
4. Il medico vede l'elenco aggiornato

Scenari Alternativi

Condizione	Comportamento
La disattivazione è richiesta per errore	Il sistema richiede una conferma prima di processare

Disattivazione e non Cancellazione

Una terapia sospesa resta registrata. Cancellarla significherebbe togliere una parte della storia clinica del paziente

UC-27: Prendere in carico una Segnalazione

UC-27	
Attore primario	Medico
Attore secondario	Paziente UC-17
Requisiti	RF-30
Precondizioni	La segnalazione riguarda un paziente in cura presso il medico ed è aperta
Postcondizioni	La segnalazione verrà chiusa scomparendo dagli aperti del medico e resta visibile al paziente come presa in carico

Scenario Principale

1. Il Medico apre il pannello delle segnalazioni
2. Il sistema mostra le segnalazioni aperte dei pazienti in cura presso il medico, con nome, tipo, gravità, testo e data
3. Il medico prende in carico una segnalazione
4. Il sistema la chiude, aggiorna l'elenco e decrementa il contatore visibile nel menù

Scenari Alternativi

Condizione	Comportamento
La segnalazione è già stata chiusa	Nessun effetto

UC-28: Chat con un Paziente

UC-28	
Attore primario	Medico
Attore secondario	Paziente
Requisiti	RF-22, RF-23
Precondizioni	Profilo del medico completo, paziente in cura presso il medico
Postcondizioni	Il messaggio è registrato nella conversazione

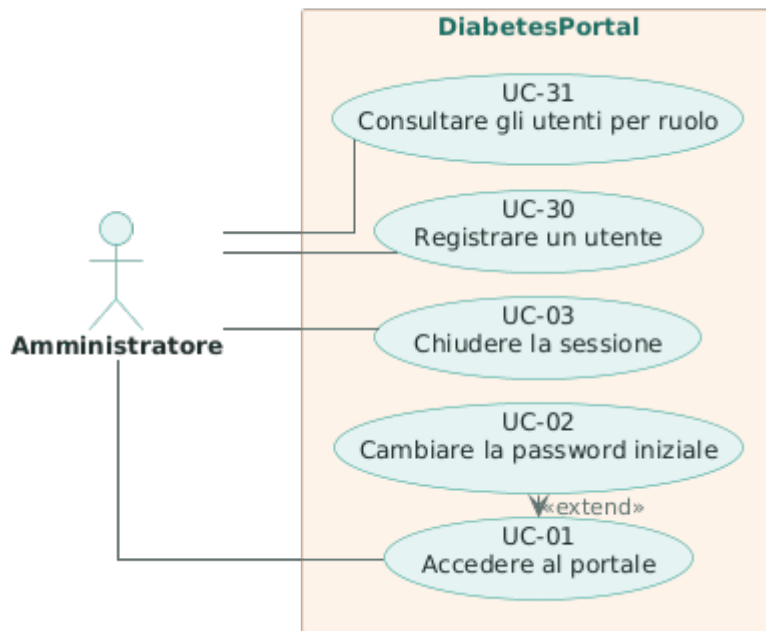
Scenario Principale

1. Il Medico apre le chat e seleziona il paziente con cui chattare dall'elenco
2. Il sistema carica gli ultimi messaggi tra il medico e il paziente
3. Il medico scrive il testo e lo invia
4. Il sistema registra il messaggio attribuendolo al medico e aggiorna la chat

Scenari Alternativi

Condizione	Comportamento
Nessun paziente	Elenco dei pazienti vuoto
Messaggio vuoto o formato da soli spazi	Messaggio rifiutato

3.6 - Amministratore



L'attore è un attore di supporto, non utilizza il portale per un proprio fine clinico, ma per rendere possibile l'accesso a chi lo ha.

UC-30: Registrazione di un Utente

UC-30	
Attore primario	Amministratore
Requisiti	RF-01
Precondizioni	Amministratore autenticato
Postcondizioni	Viene creato un nuovo utente contrassegnato come nuovo, con una password temporanea e in chiaro all'amministratore che la vedrà una sola volta

Scenario Principale

1. L'amministratore apre il modulo di registrazione e inserisce i dati: nome, cognome, indirizzo di posta e ruolo del nuovo utente
2. Il sistema verifica i campi
3. Il sistema genera una password temporanea con un generatore crittograficamente sicuro
4. Il sistema memorizza l'hash della password e crea il nuovo utente
5. Il sistema mostra all'amministratore la password in chiaro che dovrà comunicare alla persona

Scenari Alternativi

Condizione	Comportamento
Campi vuoti	Visualizzato un messaggio di errore
Indirizzo di posta già presente nel db	L'inserimento viene respinto e viene visualizzato un messaggio di errore

Password Persa

Se l'amministratore oppure l'utente perde la password temporanea, serve un intervento diretto di un amministratore che ha accesso al database.

Una futura implementazione potrebbe essere il collegamento tra la mail e il portale, in modo tale da avere un recupero password sicuro.

UC-31: Visualizzazione Utenti

UC-31	
Attore primario	Amministratore
Requisiti	RF-32
Precondizioni	Amministratore autenticato
Postcondizioni	-

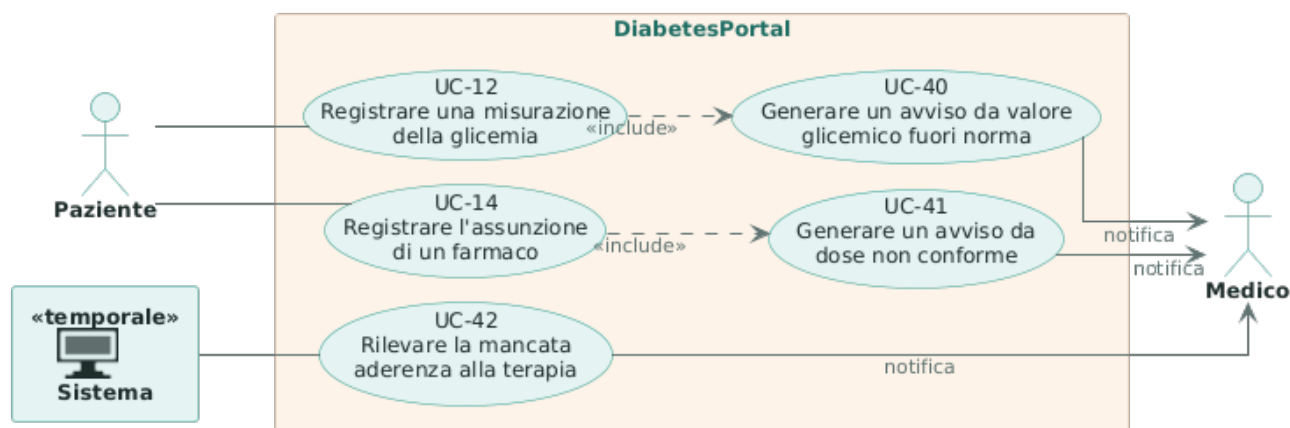
Scenario Principale

1. L'amministratore apre la pagina di visualizzazione degli utenti in base al loro ruolo, paziente o medico
2. Il sistema mostra l'elenco in ordine alfabetico mostrando il nome, cognome, mail e stato

Scenari Alternativi

Condizione	Comportamento
Elenco vuoto	Visualizzato un messaggio di elenco vuoto

3.7 - Sistema



UC-40: Generazione Avviso Glicemico

UC-40	
Attore primario	Sistema
Innesco	UC-12 registrazione di una misurazione
Requisiti	RF-13
Precondizioni	Viene registrata con successo una misurazione glicemica
Postcondizioni	Può essere creato un avviso destinato al medico di riferimento, con gravità proporzionata al caso

Scenario Principale

1. Il sistema classifica il valore registrato in base al momento del pasto che il paziente dichiara
2. Se il sistema individua una gravità, esso crea un avviso destinato al medico di riferimento del paziente con le informazioni: valore, momento, gravità e paziente

Classi di Gravità

Classe	Condizione	Gravità
Ipoglicemia severa	<54 mg/dL	Critica
Ipoglicemia	54-60 mg/dL	Attenzione
Nella norma	entro le soglie, secondo il momento	Nessuna
Iperglicemia	>130 a digiuno oppure >180 dopo un pasto	Attenzione
Iperglicemia severa	>250 mg/dL	Critica

Scenari Alternativi

Condizione	Comportamento
Il paziente non ha un medico di riferimento	L'avviso non avrebbe un destinatario, infatti c'è la configurazione obbligatoria UC-11

⚠ Soglie

Le soglie di iperglicemia sono due invece quelle di ipoglicemia è una sola, perchè un valore basso è sempre preoccupante a prescindere dal pasto.

UC-41: Generazione Avviso Assunzione

UC-41	
Attore primario	Sistema
Innesco	UC-14 registrazione di una assunzione
Requisiti	RF-17
Precondizioni	Viene registrata con successo una assunzione di un farmaco
Postcondizioni	Può essere creato un avviso destinato al medico di riferimento

Scenario Principale

1. Il sistema calcola lo scostamento fra quantità dichiarata dal paziente e quantità prescritta nella terapia
2. Se è oltre il 10% il sistema crea un avviso di gravità attenzione, se è +/-50% crea un avviso di gravità critica

Scenari Alternativi

Condizione	Comportamento
Il paziente dichiara una quantità errata	Il sistema non può distinguere il caso di errore umano, eventualmente viene creato l'avviso

UC-42: Rilevazione Mancata Aderenza alla Terapia

UC-42	
Attore primario	Sistema
Attore secondario	Medico destinatario
Innesco	Scadenza di un intervallo periodico
Requisiti	RF-31
Precondizioni	Il sistema è in esecuzione
Postcondizioni	Per ogni terapia attiva priva di assunzioni, da almeno un numero deciso di giorni, viene generato e inviato un avviso critico al medico di riferimento

Scenario Principale

1. Il sistema classifica il valore registrato in base al momento del pasto che il paziente dichiara
2. Se il sistema individua una gravità, esso crea un avviso destinato al medico di riferimento del paziente con le informazioni: valore, momento, gravità e paziente

Scenari Alternativi

Condizione	Comportamento
Nessuna terapia attiva	Il controllo termina senza fare nulla
Il paziente non ha un medico di riferimento	Viene ignorata essendo che l'avviso non avrebbe un destinatario
Esiste già un avviso dello stesso tipo	Non viene creato nessun altro avviso
Il medico prende in carico l'avviso, ma il paziente continua a non assumere il farmaco	Al controllo successivo si genera un nuovo avviso

Nessun innesco

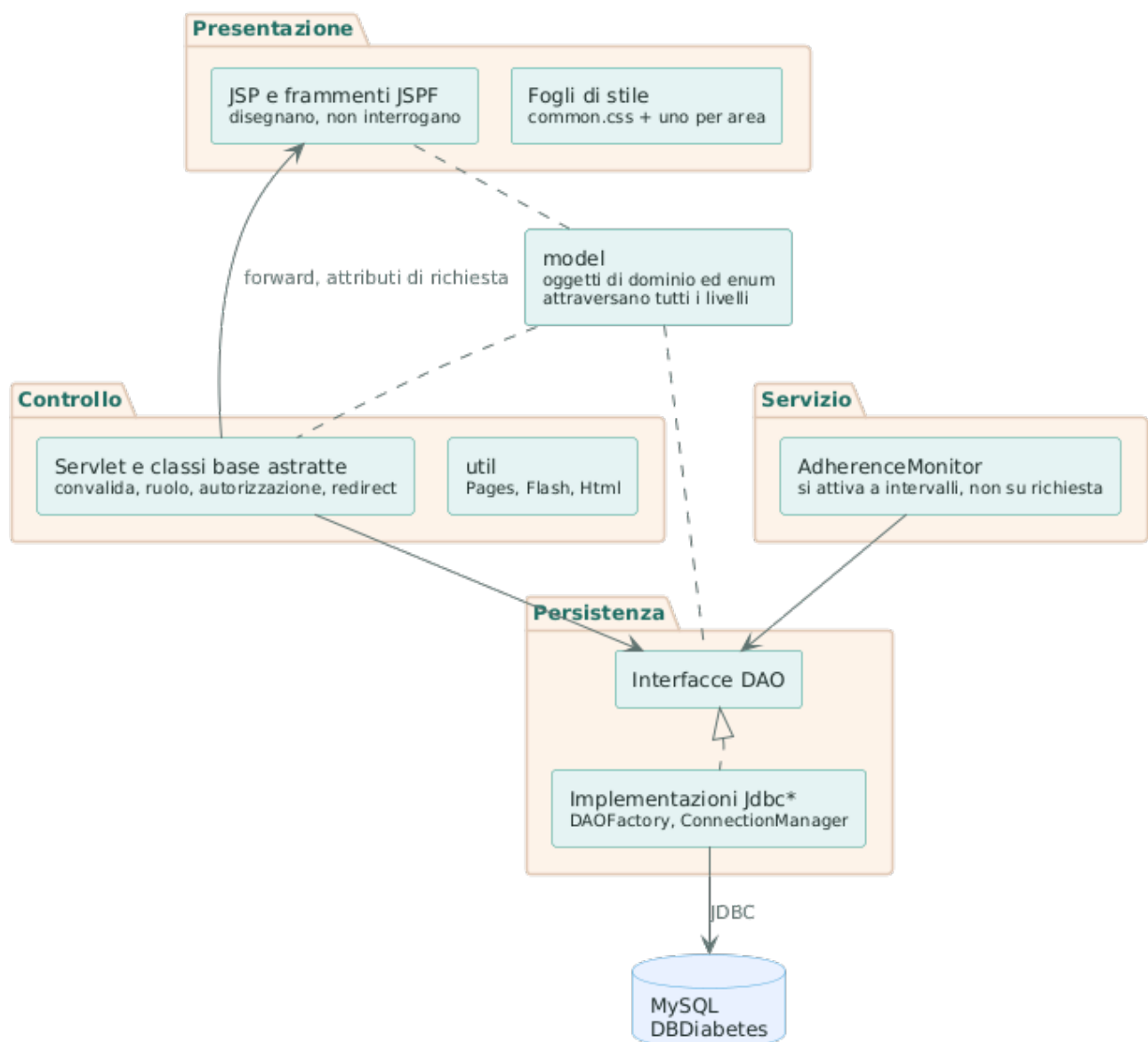
Questo use case non può essere innescato da un utente o da un medico, essendo che entrambi potrebbero non accedere al portale.

4 - Architettura

Il sistema adotta un'architettura a strati su una infrastruttura client-server a tre livelli: thin client, server applicativo e un database.

Ogni livello conosce solamente quello immediatamente sottostante, ciò implica che il livello dati è sostituibile senza toccare nient'altro e che la logica di autorizzazione ha un solo punto in cui viene applicata.

4.1 - Livelli



Presentazione

Presentazione	
Responsabilità	Produce HTML che viene elaborato dal browser per essere visualizzato dall'utente
Contiene	Pagine JSP, frammenti JSPF condivisi, fogli CSS
Dipende da	Nulla

Controllo

Controllo	
Responsabilità	Riceve una richiesta e la deve convalidare, decidere, instradare
Contiene	Servlet, classi base astratte e le utilità <code>Pages</code> , <code>Flash</code> , <code>Html</code>
Dipende da	Interfacce DAO

Query e Servlet

Le servlet non contengono nessuna query SQL, infatti queste sono contenute nelle DAO che viene richiamata in base al servizio offerto dalla pagina.

Servizio

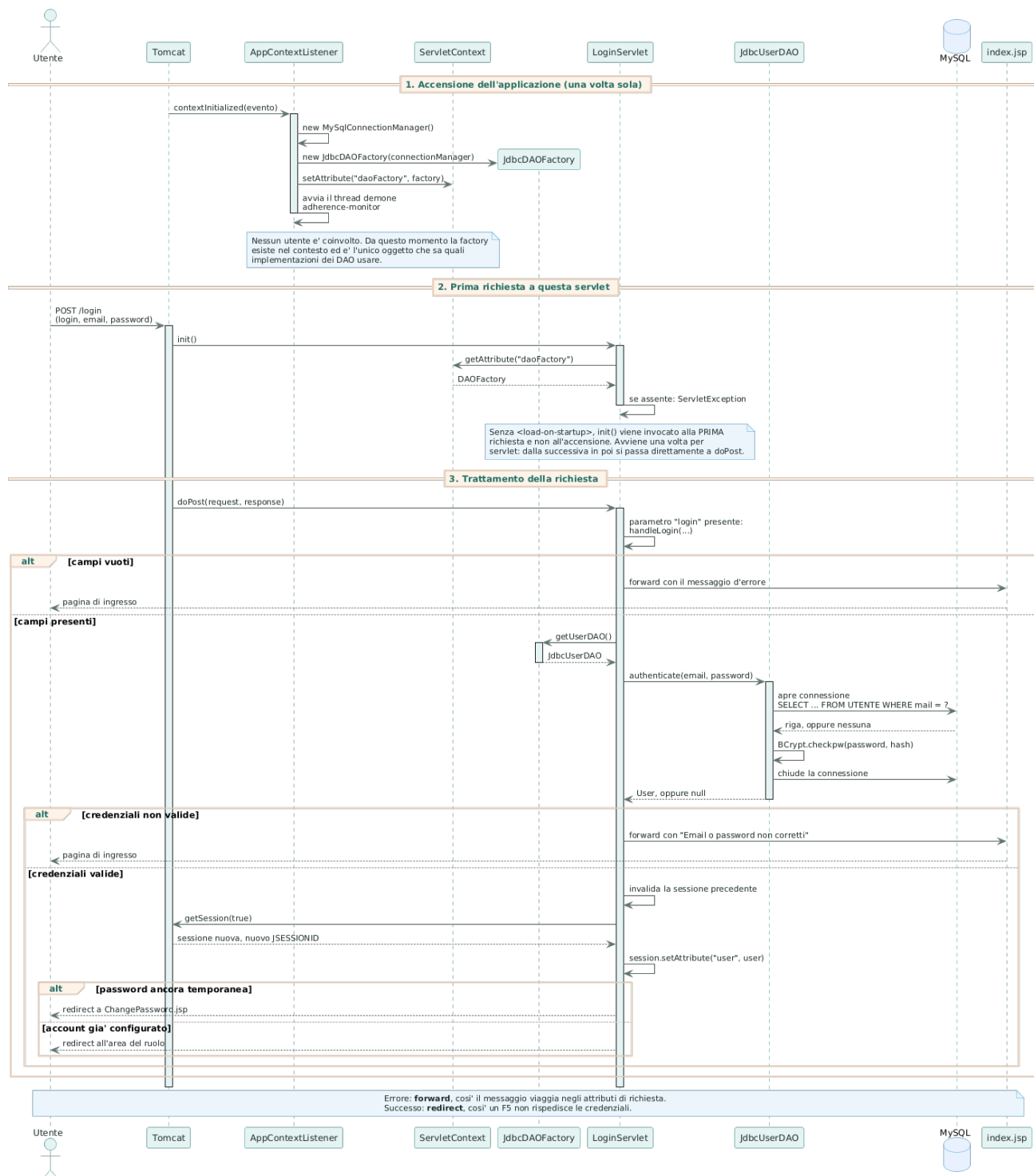
Servizio	
Responsabilità	Le operazioni effettuate dal sistema senza nessuna richiesta di un utente
Contiene	<code>AdherenceMonitor</code>
Dipende da	Interfacce DAO

Persistenza

Persistenza	
Responsabilità	Interfacciarsi con il database
Contiene	Interfacce DAO, implementazioni Jdbc, factory DAO e il gestore delle connessioni
Dipende da	Driver JDBC

4.2 - Richieste HTTP

Ogni componente del sistema entra in funzione a seguito di una richiesta HTTP, con la sola eccezione di `AdherenceMonitor`. Si vede un esempio, il caso del login, scelto perché contiene tutti i meccanismi che si ripetono nelle altre richieste.



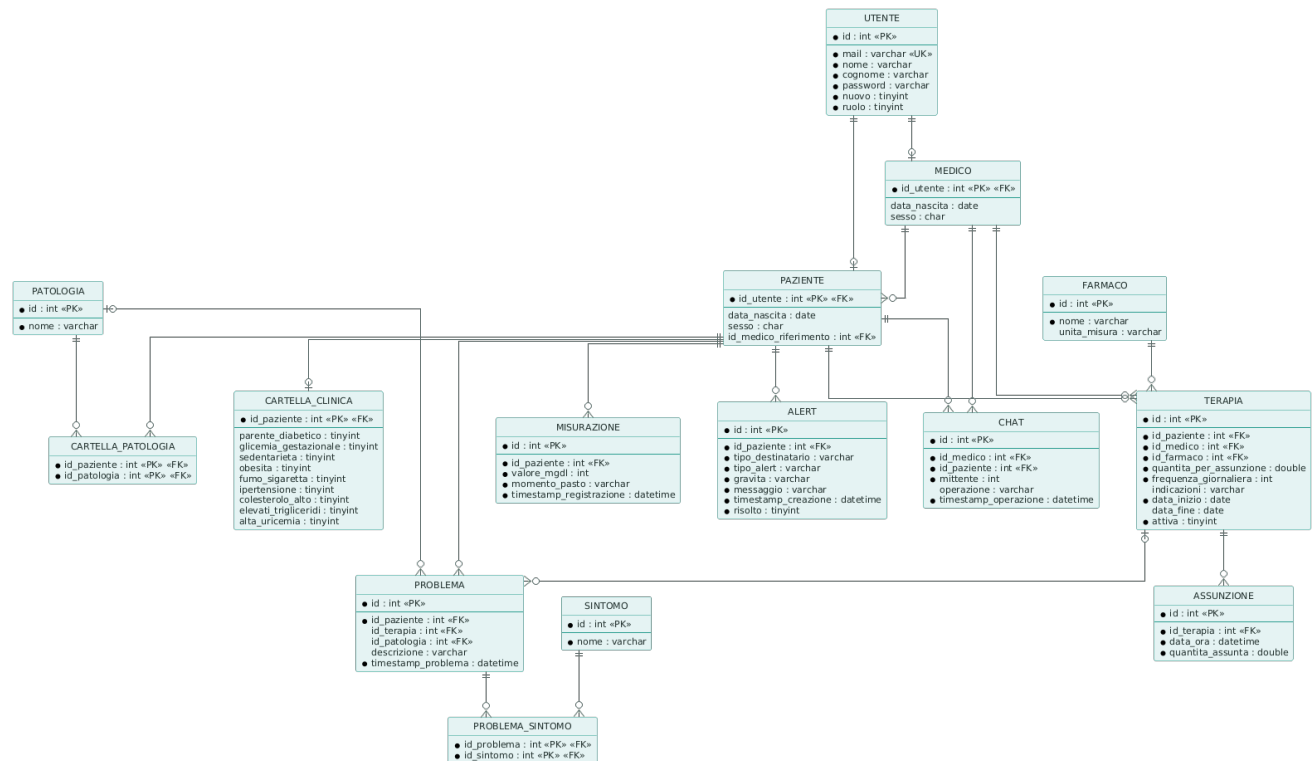
Il diagramma distingue tre momenti:

1. all'avvio dell'applicazione, una volta sola, `AppContextListener` costruisce la `JdbcDAOFactory` e la deposita nel `ServletContext`. È l'oggetto che collega il livello di controllo a quello di persistenza, ed è l'unico che sa quali implementazioni dei DAO usare
2. alla prima richiesta rivolta a una servlet, il contenitore ne invoca `init()`, che ritira la factory dal contesto, ciò avviene una volta per servlet
3. a ogni richiesta, la servlet convalida i parametri, chiede alla factory il DAO che le serve e instrada la risposta

5 - Database

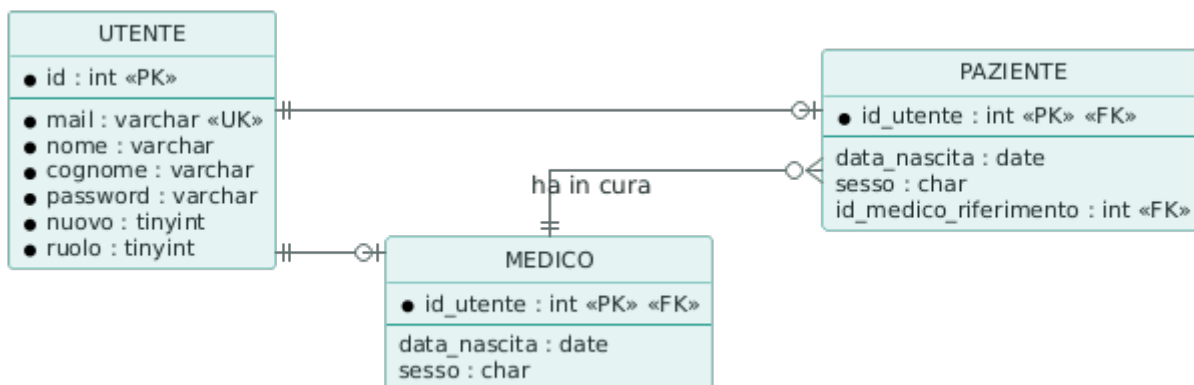
La base di dati DBDiabetes è svolta su MySQL 8. Le tabelle create saranno poi collegate in modo significativo alle classi del progetto software.

Si vede di seguito il diagramma ER della base di dati:



5.1 - Persone

Si effettua un focus sulle tabelle dedicate agli utenti registrati nel sistema.



Si notano le tabelle riguardanti gli utenti, sia PAZIENTE che MEDICO condividono la stessa chiave con UTENTE, dove sono sia chiavi primarie che chiavi esterne. Essendo chiavi primarie non ci potranno essere utenti che sono più volte pazienti e/o medici.

Inoltre tramite la tabella `UTENTE` si identifica un qualsiasi individuo all'interno del sistema, infatti tramite questa tabella viene effettuato il login.

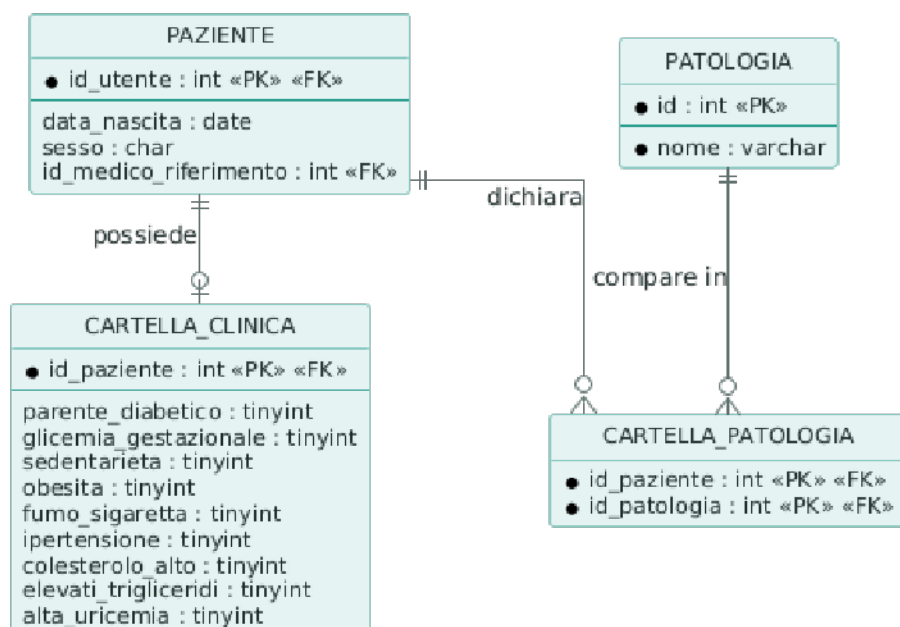
⚠ Medico che è anche paziente

Tramite questa progettazione delle tabella dedicate agli utenti del portale, un medico non può essere a sua volta un paziente tramite lo stesso `id` essendoci un campo ruolo statico.

Si presuppone che i medici associati al portale avranno anche delle mail aziendali distribuite dall'organizzazione medica stessa, in questo modo se un medico vuole essere un paziente della stessa organizzazione per cui lavora non si troverà in conflitto con il proprio indirizzo di posta che è salvato nel database in modo univoco, oltre che per motivi di sicurezza aziendali.

5.2 - Fattori di Rischio e Patologie

Si effettua un focus sulle tabella riguardanti le patologie e i fattori di rischio del paziente.



La tabella dedicata alla cartella clinica del paziente, `CARTELLA_CLINICA`, è composta dai fattori di rischio, i quali sono identificati da attributi booleani. Poiché i fattori di rischio sono contenuti tutti in una unica tabella saranno sempre chiusi e stabili, in questo modo saranno poi leggibili tramite una unica `SELECT` tramite l' `id` del paziente.

🔗 Tabella Fattori di Rischio

Si potrebbe creare una tabella `FATTORE_RISCHIO` la quale è collegata tramite una relazione `(0,N)` a una tabella `CARTELLA_FATTORI`.

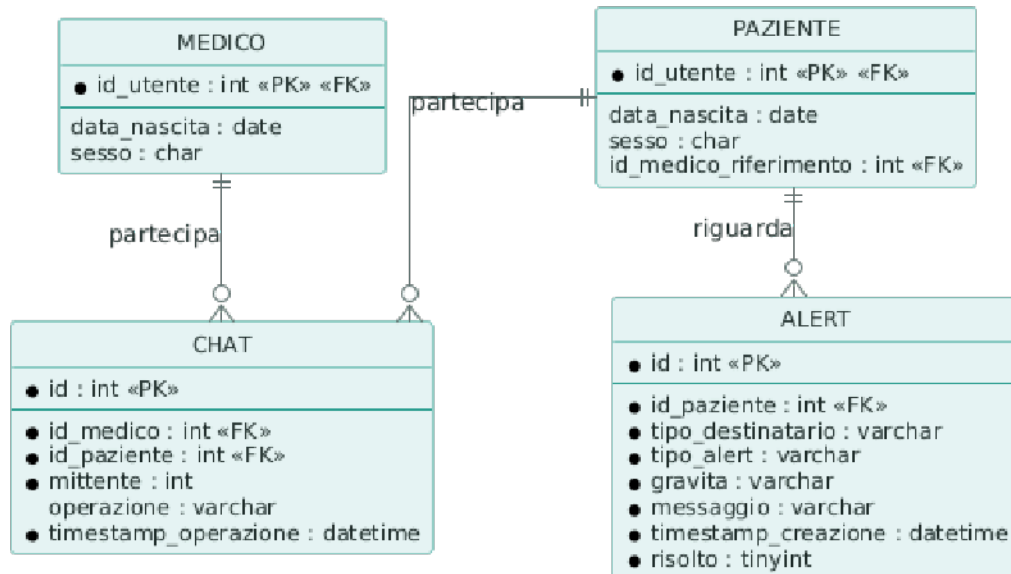
Questo metodo creerebbe una facilitazione di aggiunte di fattori di rischio, ma richiederebbe un `JOIN` per avere tutti i fattori di rischio che un paziente possiede.

Le patologie invece possono essere molteplici in base al paziente, si è deciso di inserire nel database la tabella `PATOLOGIA` contenente le patologie più frequenti per persone con il diabete di tipo 2, ma proprio essendo una tabella a sè si possono aggiungere nuove patologie in modo semplice.

🔗 Tabella FARMACO

Lo stesso ragionamento lo si è apportato sulla tabella `FARMACO`, la quale contiene solamente i farmaci più comuni prescritti a pazienti con il diabete di tipo 2, ma che si può allargare in modo semplice.

5.3 - Chat e Alert



Per la tabella `CHAT` si ha un attributo `operazione`, largo 255 caratteri, il quale contiene il messaggio scambiato.

Si hanno entrambe le chiavi degli utenti, sia del medico che del paziente, con la ripetizione della chiave nell'attributo `mittente` per identificare chi ha scritto e spedito il messaggio.

Differente è per la tabella `ALERT`, la quale deve essere consultata dal medico, ma gli alert riguardano principalmente un paziente, quindi si è scelto di inserire unicamente come chiave esterna `id_paziente`. In tale modo il medico dovrà effettuare una operazione di `JOIN` sui propri pazienti per avere gli alert attivi.

5.4 - Mappatura

Le tabelle della base di dati sono poi state mappate sulle classi Java nel seguente modo:

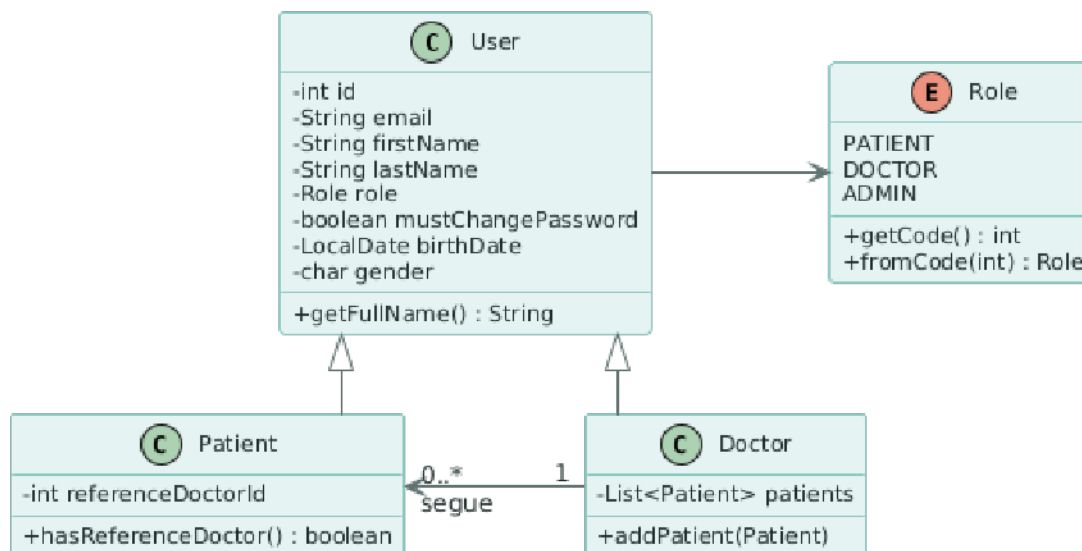
Tabella DB	Classe Java	Note attributi
UTENTE	User	ruolo è un intero identificato da una classe enum
UTENTE + PAZIENTE	Patient	Si effettua un JOIN tra le tabelle dopo aver identificato il ruolo dell'utente
UTENTE + MEDICO	Doctor	
CARTELLA_CLINICA + CARTELLA_PATOLOGIA	MedicalRecord	
MISURAZIONE	Measurement	momento_pasto è identificato tramite la classe enum MealMoment
TERAPIA + FARMACO	Therapy	Si ha l'oggetto annidato Drug
FARMACO	Drug	
ALERT	Alert	
CHAT	Message	
PATOLOGIA , SINTOMO	Pathology , Symptom	

6 - Progettazione

6.1 - Dominio

Il package `model` contiene 16 classi, raggruppabili in quattro sottogruppi.

Persone



La classe `Patient` e `Doctor` derivano da `User` rispecchiando il database.

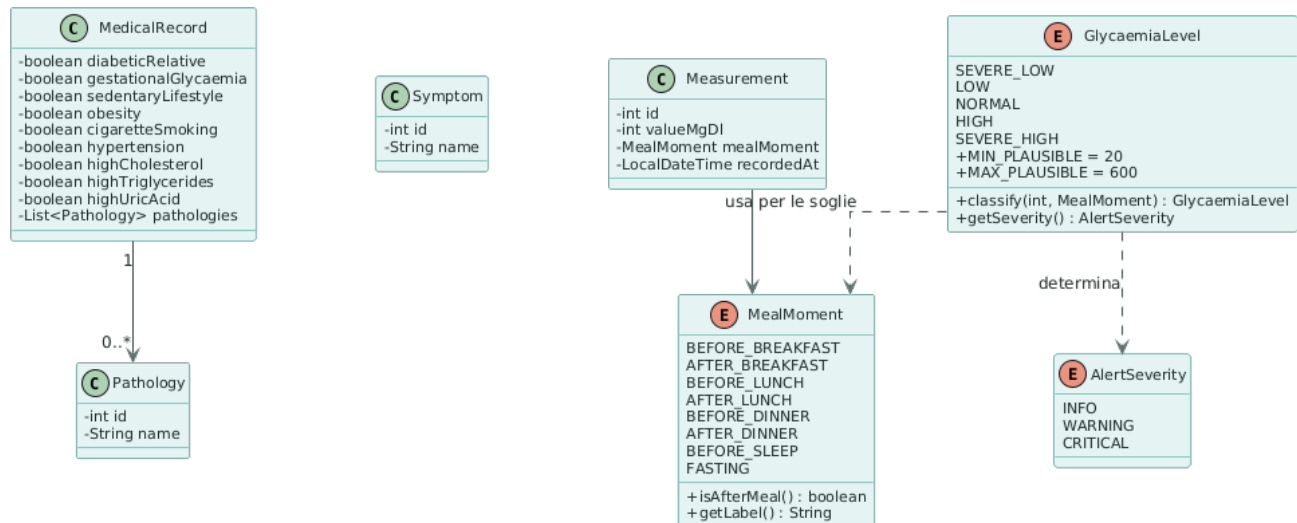
Il costruttore del paziente e/o del medico creano una loro istanza a partire dall'utente già letto.

Costruttore classe `Patient`

```
public Patient(User, LocalDate, char);
```

La derivazione porta che un `Patient` è un `User`, quindi può stare in sessione come utente generico.

Dati Clinici

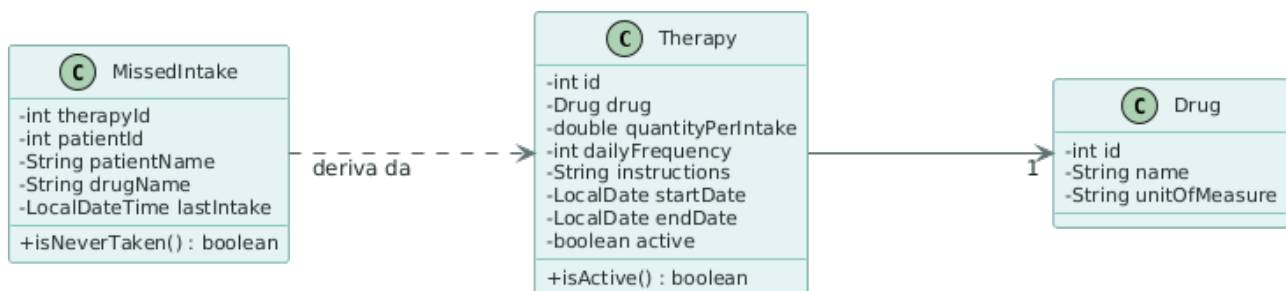


MedicalRecord ha nove booleani, come si vede anche nel database, se in un futuro si vorrebbe cambiare questo punto bisognerà inserire una lista di **Fattori di rischio**.

Tramite i differenti enum si calcolano se i valori che il paziente inserisce portano ad eventuali alert, con annessa gravità.

Inoltre la classe **Symptom** non ha collegamenti, essendo che il legame tra sintomi e problemi vive solo sul database nella tabella **PROBLEMA_SINTOMO**.

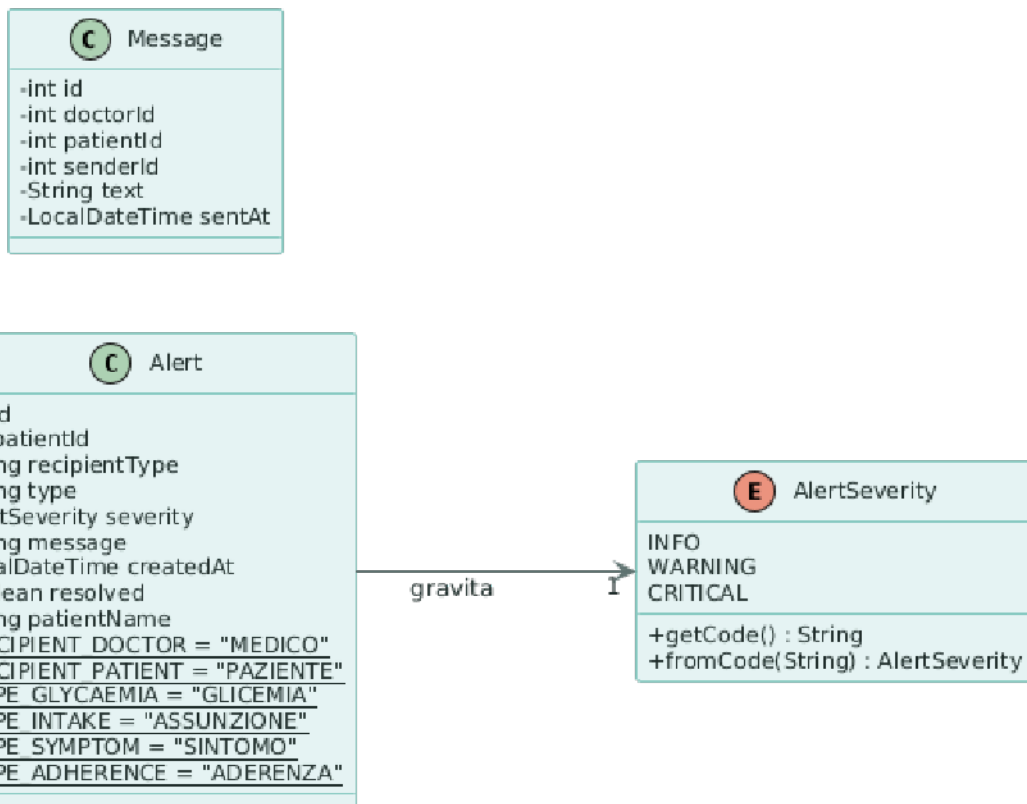
Terapie



In questo sottogruppo si può notare una differenza rispetto al database, infatti la classe **Therapy** contiene un oggetto **Drug** e non un identificativo.

Si nota la classe **MissedIntake**, la quale non ha una tabella corrispondente nel db. Questa tabella nasce dal controllo se i pazienti rispettano le terapie attive.

Comunicazione



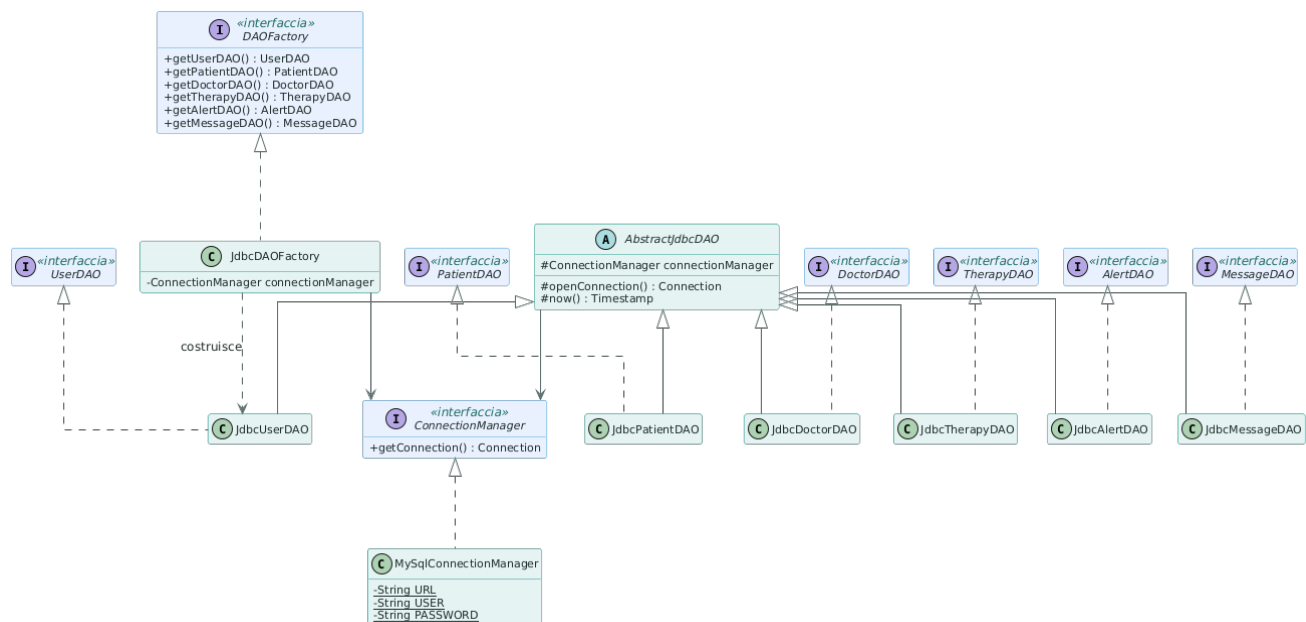
Nella classe **Alert** sono presenti sei costanti, le quali indicano poi nel database il destinatario dell'alert e la tipologia di alert che viene creata/memorizzata.

La classe **Message** invece segue la tabella **CHAT** della basi di dati.

Destinatario Paziente

Al momento nel sistema non esiste un alert dove il destinatario è il paziente, ma si suppone che in una futura release ci siano alert, ad esempio pubblicitarie di servizi della società medica, destinati verso il paziente

6.2 - Persistenza



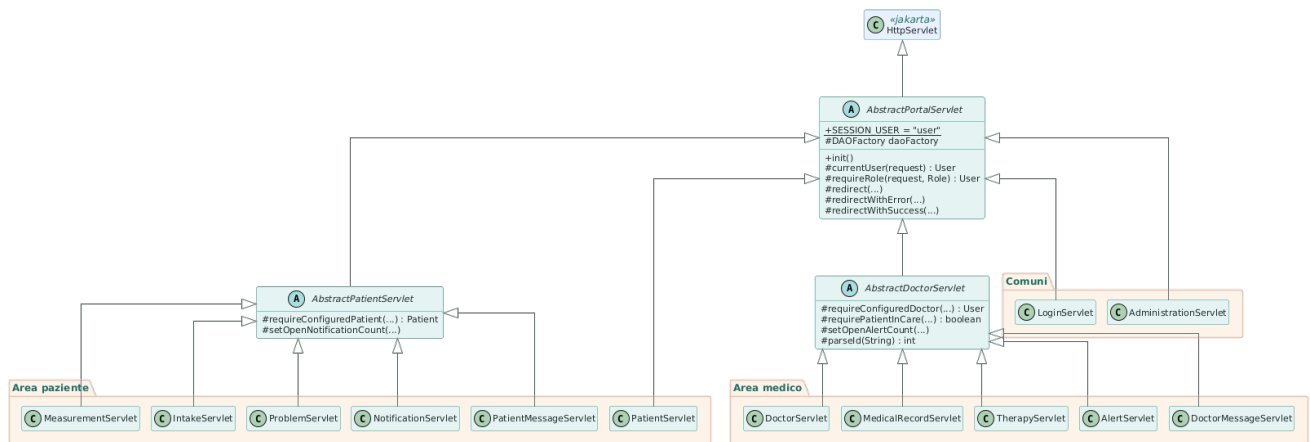
La `JdbcDAOFactory` distribuisce le diversi DAO, le quali anche queste sono interfacce poi modellate tramite le classi `Jdbc`.

La classe astratta `AbstractJdbcDAO` contiene due soli metodi: `openConnection` e `now`, i quali aprono la connessione e restituiscono il timestamp del momento in cui viene richiamato.

Le responsabilità delle DAO sono divise in base a un determinato criterio:

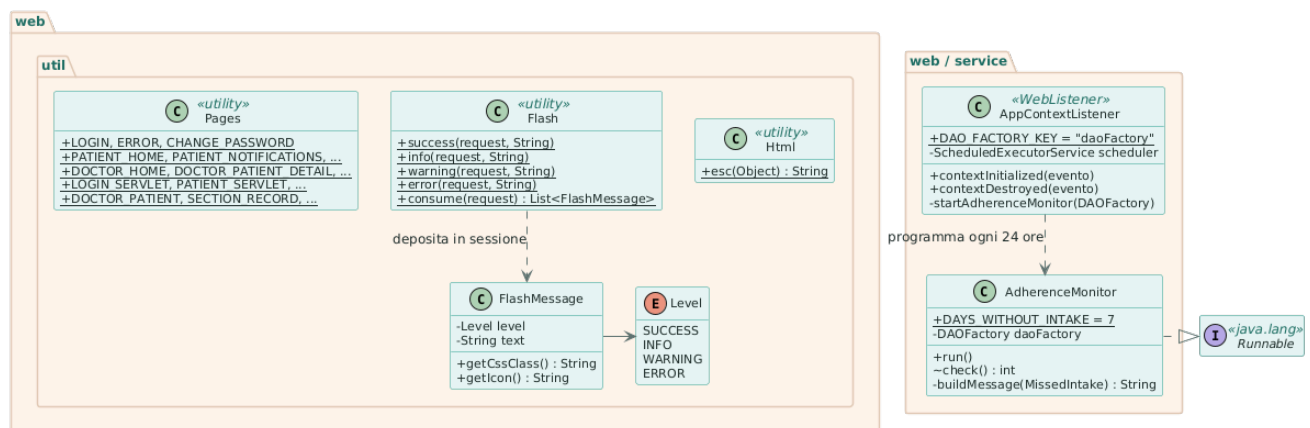
DAO	Tabella	Criterio
UserDAO	UTENTE	Account e credenziali
PatientDAO	PAZIENTE ,	Tutto ciò che il paziente produce o dichiara di se stesso
DoctorDAO	MEDICO , PAZIENTE	Il Medico e il suo elenco di pazienti
TherapyDAO	TERAPIA , FARMACO , ASSUNZIONE	Atti medici
AlertDAO	ALERT	Avvisi
MessageDAO	CHAT	Messaggi

6.3 - Controllo



Il livello di controllo è formato da 13 servlet, in base al servizio offerto, e tre classi astratte. Si è deciso di utilizzare delle classi astratte perchè ci sono dei componenti comuni e ripetuti, per esempio la presa della factory e i controlli.

6.4 - Utilità e Servizio



La classe `Pages` raccoglie tutti i percorsi in un solo file, andando a semplificare la comprensione dei reindirizzamenti dell'utente.

La classe `Flash` deposita i messaggi in sessione e il metodo `consume` li restituisce e li cancella.

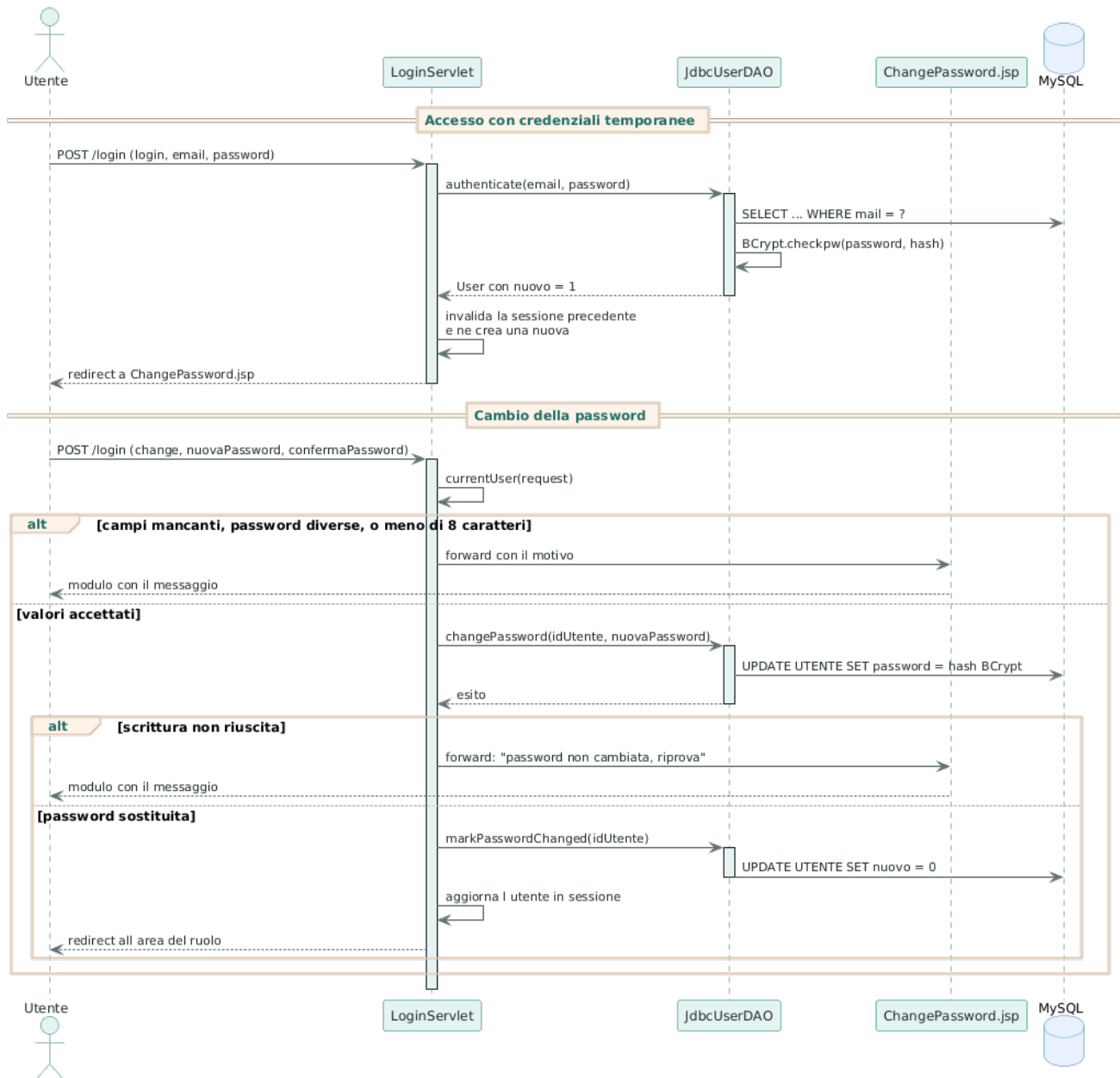
Tramite la classe `Html` con il suo metodo `esc` fanno da difesa contro l'XSS del portale.

`AppContextListener` costruisce la `DAOFactory` e la deposita nel `ServletContext`, da dove poi ogni servlet la recupera tramite `init`. Avvia poi il compito periodico `AdherenceMonitor` su un thread dedicato.

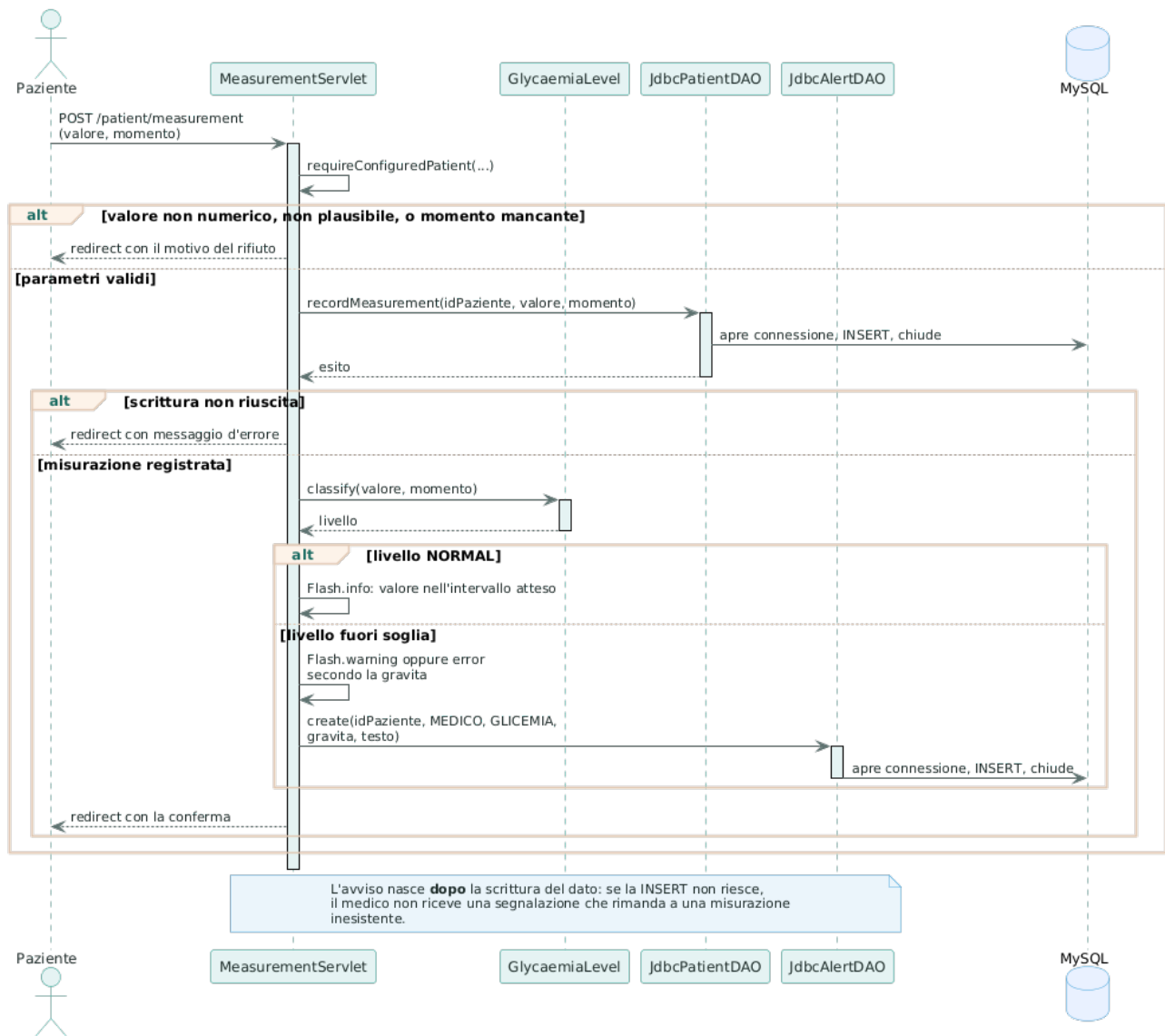
7 - Scenari di Interazioni

Si vedono ora come vengono gestite alcuni dei comportamenti comuni degli utenti che vengono effettuati dal/al sistema.

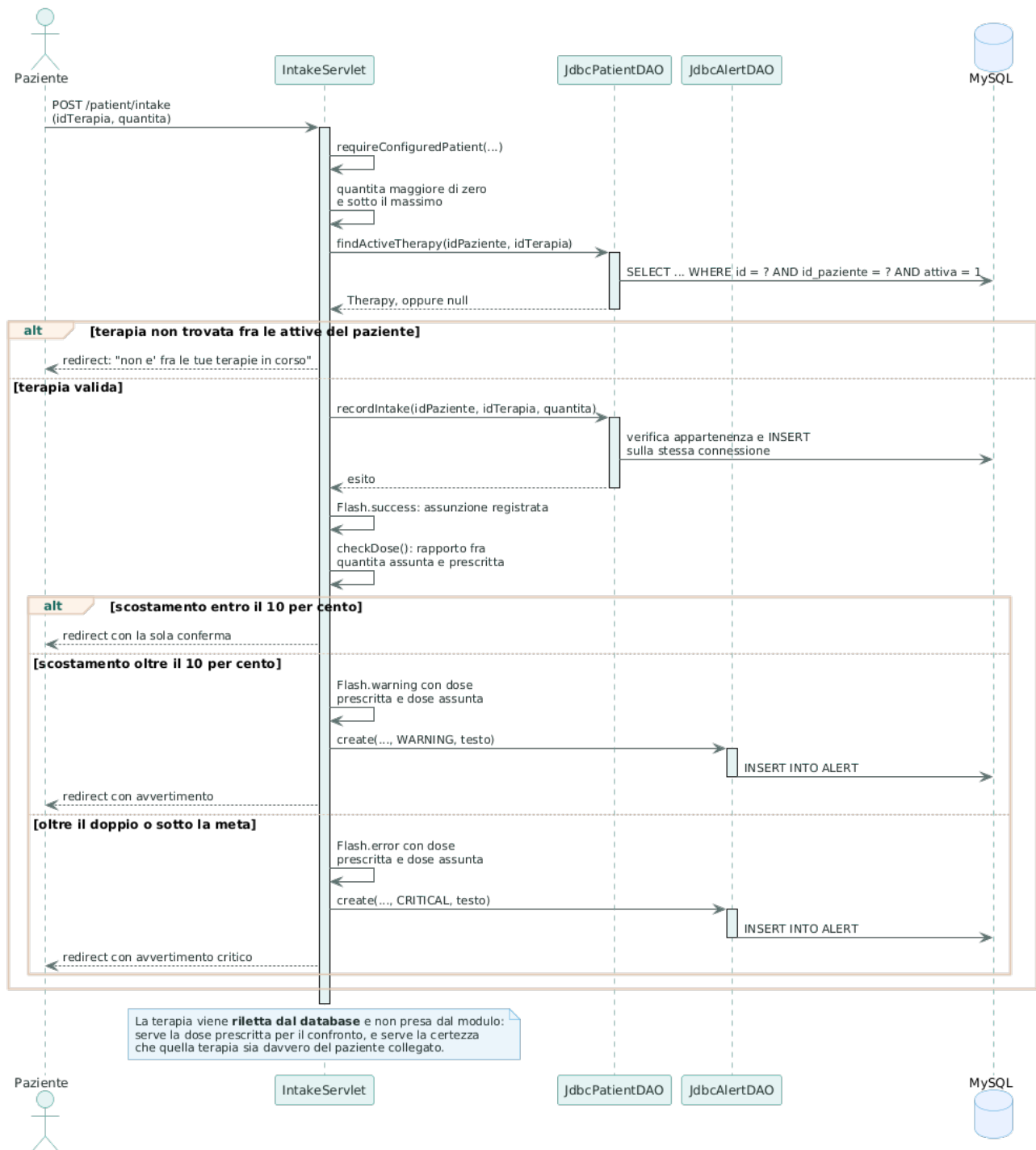
7.1 - Accesso e Cambio Password



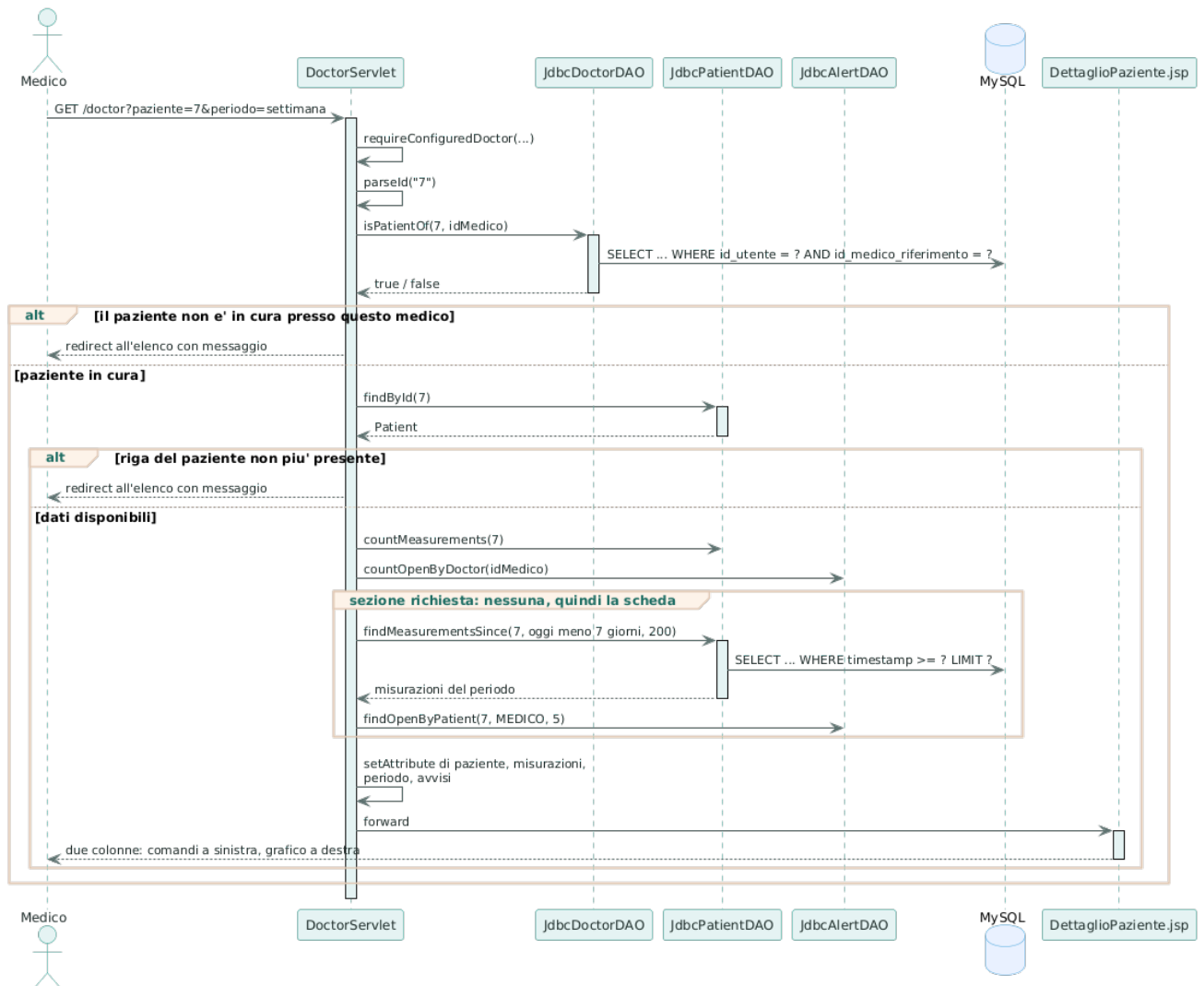
7.2 - Registrazione di una Misurazione



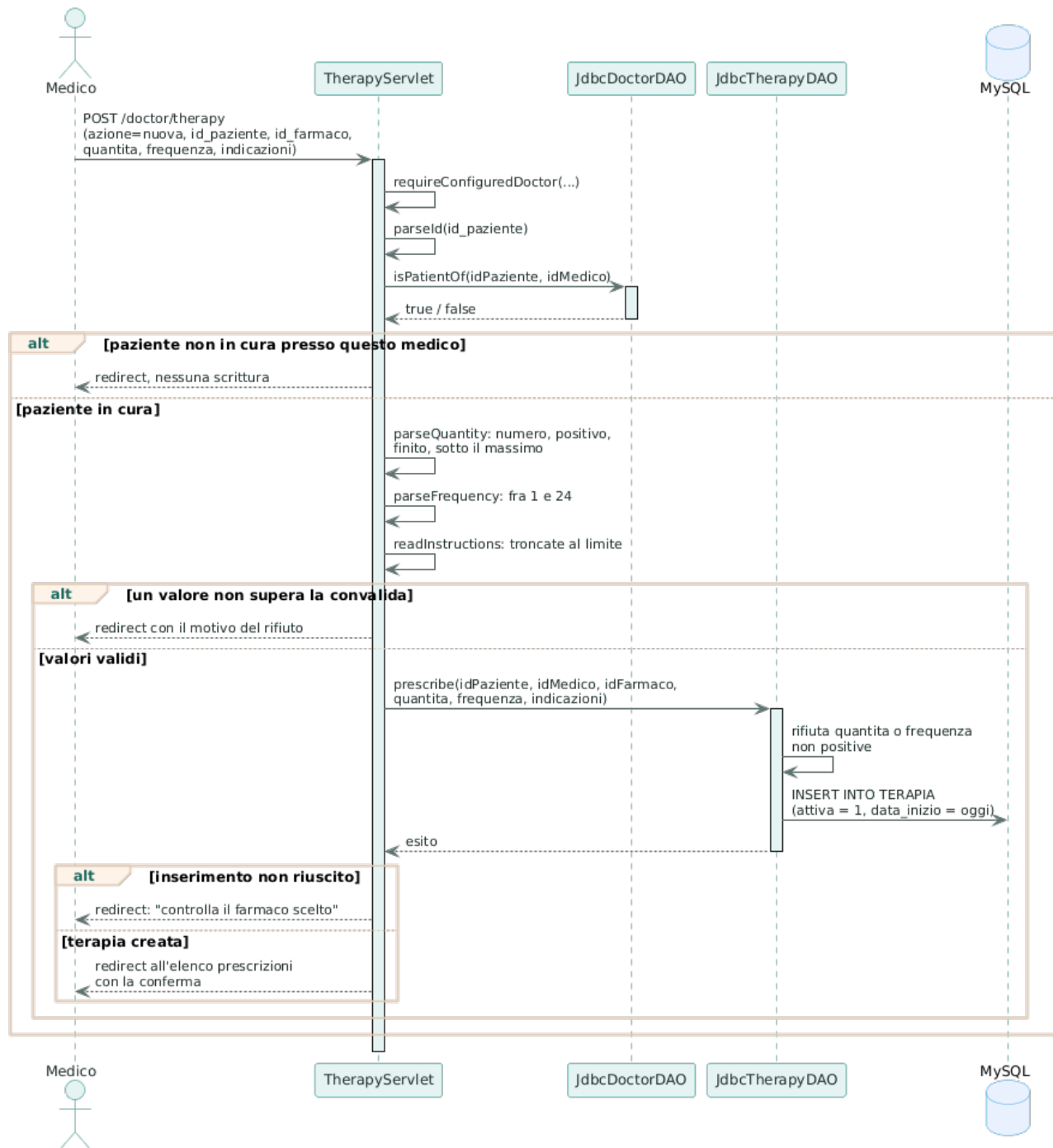
7.3 - Registrazione di un'Assunzione



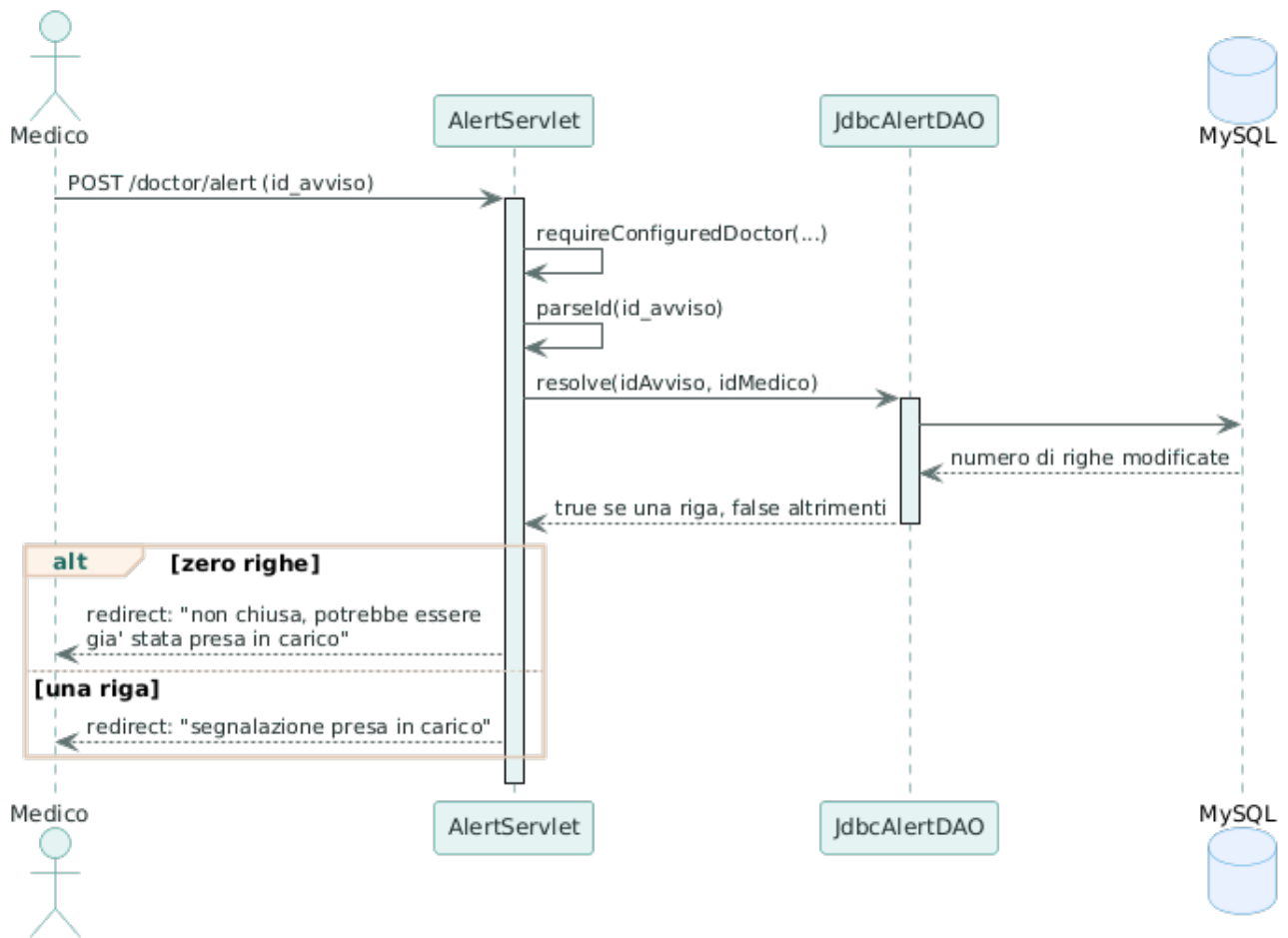
7.4 - Medico Apre i Dettagli di un Paziente



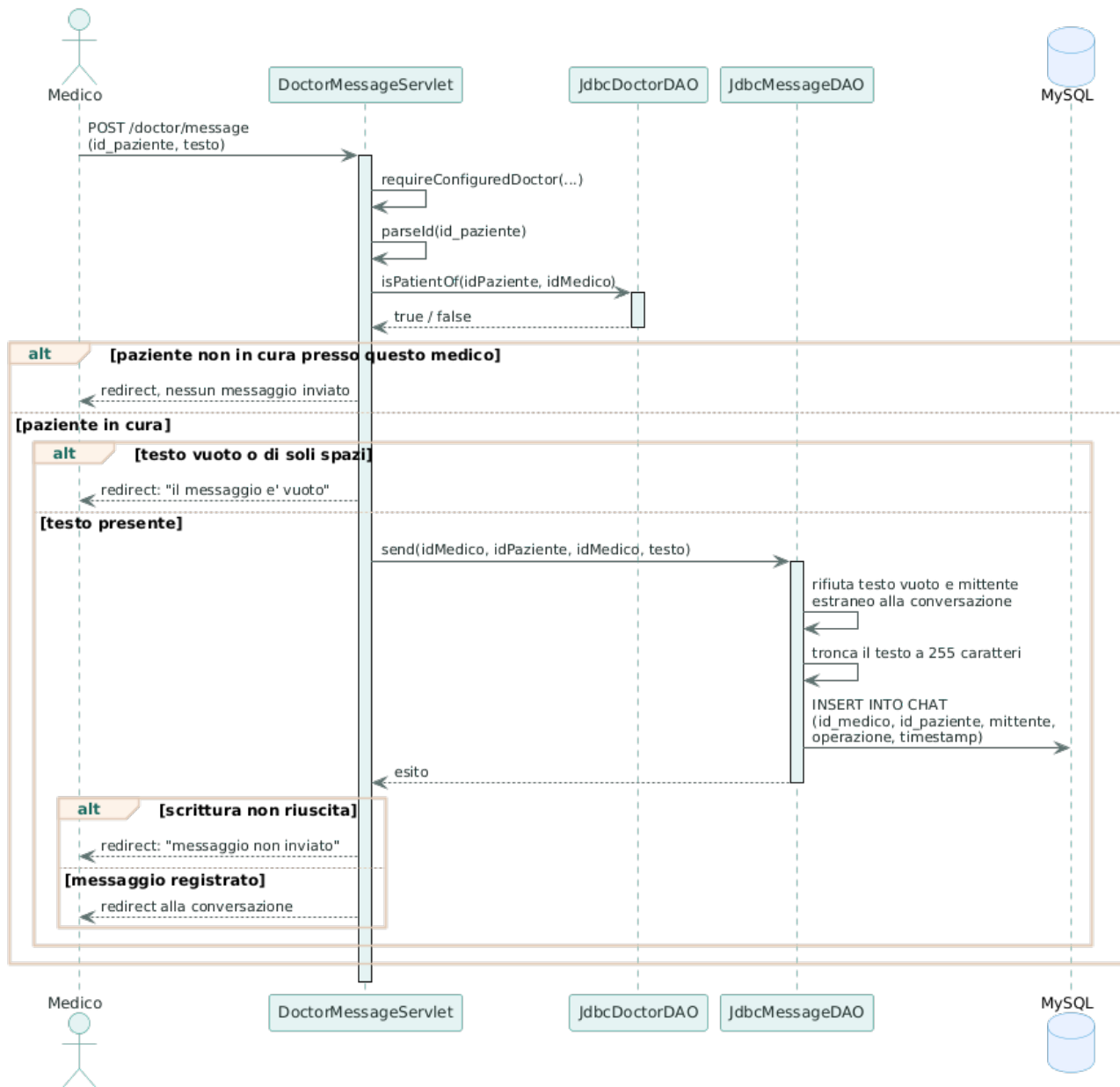
7.5 - Medico Prescrive una Terapia



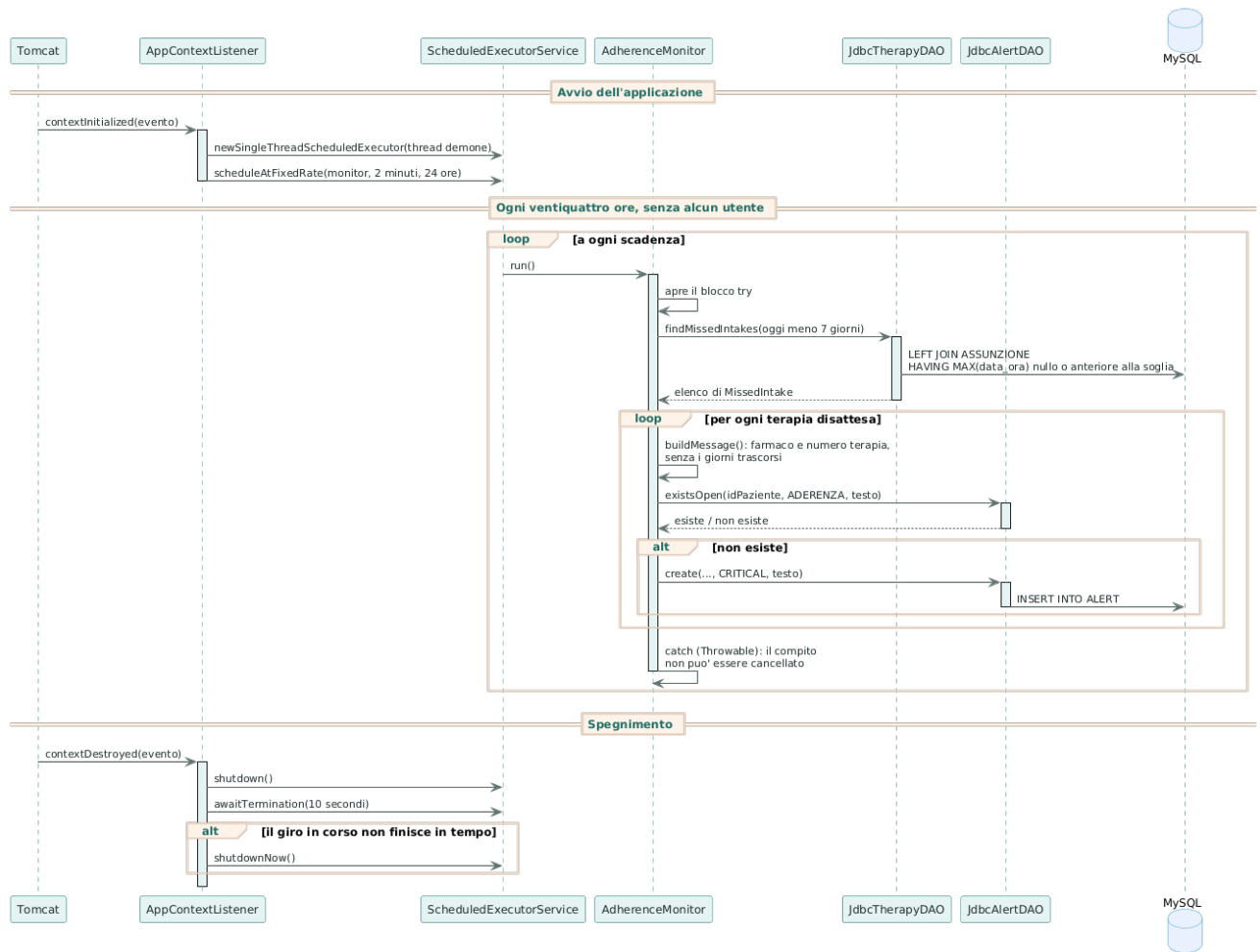
7.6 - Presa a carico di una Segnalazione



7.7 - Invio di un Messaggio



7.8 - Controllo Terapie



8 - Package

Si vedono dei blocchi di codice significativi per la comprensione all'interno delle differenti classi dei package.

8.1 - Package model

User, Patient e Doctor

User porta i capi della tabella `UTENTE` del database con l'aggiunta della data di nascita e sesso, informazioni che vengono memorizzate in `PAZIENTE / MEDICO`.

Le classi `Patient` e `Doctor` derivano da `User` con un costruttore che parte da un utente già letto nel sistema:

```
public Patient(User user, LocalDate birthDate, char gender){
    super(user.getId(), user.getEmail(), user.getFirstName(),
          user.getLastName(), user.getRole(),
          user.isMustChangePassword());
    setBirthDate(birthDate);
    setGender(gender);
}
```

La classe `Patient` ha una variabile `referenceDoctorId`, la quale indica l'identificativo del proprio medico di base. Per il controllo che il paziente abbia un medico si usa il metodo `hasReferenceDoctor` in modo tale da non creare eventuali errori avendo un identificativo pari a zero `0`.

Role

Enum per identificare il ruolo di un utente nel sistema.

```
PATIENT(0), DOCTOR(1), ADMIN(2)
```

Si usano poi i metodi `getCode()` per scrivere sul database il ruolo, restituendo il numero identificativo, e `fromCode(int)` per leggere un codice nel ruolo identificato.

Se `fromCode(int)` non rileva nessun ruolo perchè il codice passato è fuori dal range restituisce `null`, dove poi la servlet leggendo il valore nullo invalida la sessione.

MealMoment

Enum con gli otto valori riguardanti il momento della giornata in cui si effettua la misurazione glicemica.

```
BEFORE_BREAKFAST("Prima di colazione", false),
AFTER_BREAKFAST("Dopo colazione", true),
BEFORE_LUNCH("Prima di pranzo", false),
AFTER_LUNCH("Dopo pranzo", true),
BEFORE_DINNER("Prima di cena", false),
AFTER_DINNER("Dopo cena", true),
BEFORE_SLEEP("Prima di coricarsi", false),
FASTING("A digiuno", false);
```

Si ha un composto di una etichetta e un booleano, il booleano identifica poi la soglia in `GlycaemiaLevel`.

AlertSeverity

Enum formato da `INFO`, `WARNING`, `CRITICAL` con un codice testuale, i quali identificano la gravità dell'alert. Sono in coppia con un codice testuale perchè sul database si è deciso che la gravità sia una stringa, con in futura possibilità di aggiungere differenti gravità.

GlycaemiaLevel

Enum che contiene le classi di soglie della misurazione glicemica classificando il valore inserito dall'utente. In base alla soglia in cui si trova è associato l'alert adeguato.

```
SEVERE_LOW("Ipoglicemia severa", AlertSeverity.CRITICAL),
LOW("Ipoglicemia", AlertSeverity.WARNING),
NORMAL("Nella norma", null),
HIGH("Iperglicemia", AlertSeverity.WARNING),
SEVERE_HIGH("Iperglicemia severa", AlertSeverity.CRITICAL);
```

```
private static final int SEVERE_LOW_THRESHOLD = 54;
private static final int LOW_THRESHOLD = 70;
private static final int HIGH_FASTING_THRESHOLD = 130;
private static final int HIGH_AFTER_MEAL_THRESHOLD = 180;
private static final int SEVERE_HIGH_THRESHOLD = 250;
```

```

public static GlycaemiaLevel classify(int valueMgDl, MealMoment moment) {
    if (valueMgDl < SEVERE_LOW_THRESHOLD) {
        return SEVERE_LOW;
    }
    if (valueMgDl < LOW_THRESHOLD) {
        return LOW;
    }

    boolean afterMeal = (moment != null) && moment.isAfterMeal();
    int highThreshold = afterMeal ? HIGH_AFTER_MEAL_THRESHOLD :
        HIGH_FASTING_THRESHOLD;
    if (valueMgDl <= highThreshold) {
        return NORMAL;
    }
    if (valueMgDl <= SEVERE_HIGH_THRESHOLD) {
        return HIGH;
    }
    return SEVERE_HIGH;
}

```

MissedIntake

La classe della mancata aderenza di un paziente alla propria terapia attiva, è l'unica classe senza tabella associata nel database.

La classe è composta dalle seguenti variabili:

```

private final int therapyId;
private final int patientId;
private final String patientName;
private final String drugName;
private final LocalDateTime lastIntake;

```

Tutte le variabili servono per creare il messaggio e l'alert che è associato al paziente.

8.2 - Package persistence

MySQLConnectionManager

Tramite questa classe si risolvono le credenziali una unica volta tramite un blocco statico eseguito al primo uso della classe.

```
static {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
    } catch (ClassNotFoundException e) {}

    Properties props = new Properties();
    try (InputStream in =
MySQLConnectionManager.class.getClassLoader()
        .getResourceAsStream(CONFIG_FILE)) {

        if (in != null) {
            props.load(in);
        }
    } catch (IOException e) {
        // props resta vuoto e valgono i valori di default.
    }

    URL = props.getProperty("db.url", DEFAULT_URL);
    USER = props.getProperty("db.user", DEFAULT_USER);
    PASSWORD = props.getProperty("db.password", DEFAULT_PASSWORD);
}
```

Inizialmente nel blocco statico si caricano i driver MySQL, poi si cerca `db.properties` sul classpath, nei quali sono contenute le credenziali verso il db.

Se `db.properties` è assente si useranno dei dati di default, infatti si cerca il file e lo si mappa tramite chiave valore, poi alla creazione delle costanti se non trova la chiave verrà messo il valore di default.

La connessione verrà poi stabilita tramite il `getConnection()`.

JdbcDAOFactory

Questa classe viene costruita una volta in `AppContextListener` e messa nel contesto dell'applicazione. Ogni servlet la recupera tramite `init()` e attraverso la classe si procura i differenti DAO per effettuare le interrogazioni al db.

8.3 - Package web

AbstractPortalServlet

Viene dichiarato al suo interno il metodo `init()`, il quale viene chiamato una volta per servlet.

```
@Override
public void init() throws ServletException {
    Object attribute = getServletContext()
        .getAttribute(AppContextListener.DAO_FACTORY_KEY);
    if (!(attribute instanceof DAOFactory)) {
        throw new ServletException("DAOFactory assente");
    }
    this.daoFactory = (DAOFactory) attribute;
}
```

Inoltre si controlla se l'utente è autenticato, inoltre controlla che richieda un servizio riguardante il ruolo corretto da lui posseduto.

```
protected HttpSession currentSession(HttpServletRequest request) {
    return request.getSession(false);
}

protected User currentUser(HttpServletRequest request) {
    HttpSession session = currentSession(request);
    if (session == null) {
        return null;
    }
    Object user = session.getAttribute(SESSION_USER);
    return (user instanceof User) ? (User) user : null;
}

protected User requireRole(HttpServletRequest request, Role required) {
    User user = currentUser(request);
    if (user == null) {
        return null;
    }
    if (user.getRole() != required) {
        return null;
    }
    return user;
}
```

Come prima cosa si controlla se esiste già una sessione, tramite `getSession(false)`. Se non è presente nessuna sessione allora l'utente non si è autenticato. Poi viene effettuato il controllo se l'utente esiste e se possiede il ruolo richiesto.

AbstractPatientServlet e AbstractDoctorServlet

In entrambe le servlet astratte viene verificato se l'utente che richiede il servizio ha il ruolo adatto per richiederlo, inoltre verifica se è stata portata a termine la configurazione iniziale.

```
protected Patient requireConfiguredPatient(HttpServletRequest request,
                                             HttpServletResponse response) throws IOException {

    User user = requireRole(request, Role.PATIENT);
    if (user == null) {
        redirect(request, response, Pages.LOGIN);
        return null;
    }

    HttpSession session = request.getSession(true);
    Object inSession = session.getAttribute(SESSION_USER);

    Patient patient = (inSession instanceof Patient)
        ? (Patient) inSession
        : daoFactory.getPatientDAO().findById(user.getId());

    if (patient == null) {
        redirect(request, response, Pages.PROFILE_DATA);
        return null;
    }
    if (!patient.hasReferenceDoctor()) {
        redirect(request, response, Pages.PATIENT_SERVLET);
        return null;
    }

    session.setAttribute(SESSION_USER, patient);
    setOpenNotificationCount(request, patient);
    return patient;
}
```

LoginServlet

LoginServlet è l'unica servlet che lavora prima che esista un ruolo definito per l'utente, per questo motivo deriva direttamente da AbstractPortalServlet.

Questa servlet gestisce sia il modulo di login, che il modulo di cambio password iniziale.

In base alla richiesta effettuata si viene demandati alla sotto-funzione di riferimento:

```
@Override
protected void doPost(HttpServletRequest request,
                        HttpServletResponse response) throws ServletException,
IOException {

    if (request.getParameter("login") != null) {
        handleLogin(request, response);
        return;
    }
    if (request.getParameter("change") != null) {
        handlePasswordChange(request, response);
        return;
    }
    redirect(request, response, Pages.LOGIN);
}
```

Quando si richiede di fare il login si effettua una query al database per avere l'utente, se l'utente non esiste oppure le credenziali non sono corrette verrà mostrato un messaggio di errore identico.

```
User user = userDao.authenticate(email, password);
if (user == null) {
    forwardToLoginWithError(request, response,
        "Email o password non corretti.");
    return;
}
HttpSession previous = currentSession(request);
if (previous != null) {
    previous.invalidate();
}

HttpSession session = request.getSession(true);
session.setAttribute(SESSION_USER, user);
```

Inoltre ogni volta che si effettua il login si invalida la sessione precedente e se ne crea una nuova con i nuovi dati dell'utente.

Se l'utente è la prima volta che accede al portale verrà reindirizzato alla pagina di cambio password.

```
if (user.isMustChangePassword()) {  
    redirect(request, response, Pages.CHANGE_PASSWORD);  
    return;  
}
```

In caso contrario l'utente verrà indirizzato verso la propria area in base al ruolo.

```
private void routeByRole(HttpServletRequest request,  
    HttpServletResponse  
        response, HttpSession session, User user) throws IOException  
{  
  
    Role role = user.getRole();  
    if (role == null) {  
        session.invalidate();  
        redirect(request, response, Pages.ERROR);  
        return;  
    }  
  
    switch (role) {  
        case PATIENT:  
            redirect(request, response, Pages.PATIENT_SERVLET);  
            break;  
        case DOCTOR:  
            redirect(request, response, Pages.DOCTOR_SERVLET);  
            break;  
        case ADMIN:  
            redirect(request, response, Pages.ADMIN_SERVLET);  
            break;  
        default:  
            session.invalidate();  
            redirect(request, response, Pages.ERROR);  
            break;  
    }  
}
```

Se l'utente possiede un ruolo inesistente, come detto precedentemente, verrà reindirizzato alla pagina di errore dopo che la sessione è stata invalidata.

PatientServlet

Quando il paziente viene demandato alla propria area dal `LoginServlet`, si ritrova nel metodo `GET`, il quale fa da spargitrafico mandando l'utente nella pagina corretta in base se ha completato la sua configurazione.

Stato nella base di dati	Destinazione
Non registrato come paziente in <code>PAZIENTE</code>	<code>Data.jsp</code>
Nessun medico di riferimento scelto	<code>Patologie.jsp</code>
Configurazione completa	<code>Home.jsp</code>

Se il paziente deve ancora configurare il suo profilo, verranno effettuate delle query per avere i dati da mostrargli, per esempio la lista dei medici, prima di indirizzarlo verso la pagina.

Se la configurazione è completa vengono effettuate le query per avere i dati necessari per la schermata home del paziente.

```
private void loadHomeData(HttpServletRequest request, Patient patient)
{
    PatientDAO patientDAO = daoFactory.getPatientDAO();

    int points = normalizePoints(request.getParameter("punti"));

    request.setAttribute("terapie",
        patientDAO.findActiveTherapies(patient.getId()));

    request.setAttribute("misurazioni",
        patientDAO.findRecentMeasurements(patient.getId(), points));

    request.setAttribute("totaleMisurazioni",
        patientDAO.countMeasurements(patient.getId()));

    request.setAttribute("puntiGrafico", points);

    request.setAttribute("puntiDisponibili", CHART_POINT_OPTIONS);

    request.setAttribute("avvisi",
        daoFactory.getAlertDAO().
            findOpenByPatient(patient.getId(),
                HOME_NOTIFICATION_PREVIEW));

    request.setAttribute("notificheAperte",
        daoFactory.getAlertDAO().countOpenByPatient(patient.getId()));
}
```

Si passa anche il numero di punti del grafico dell'andamento glicemico. Il numero di punti è un elenco chiuso.

Il numero dei punti del grafico è modificabile dall'utente, per questo viene fatto un controllo se viene selezionato un numero, altrimenti si riporterà un valore di default impostato a 50 .

```
static final int[] CHART_POINT_OPTIONS = {20, 50, 100, 200};
```

```
private int normalizePoints(String raw) {  
    Integer requested = parseInteger(raw);  
    if (requested != null) {  
        for (int allowed : CHART_POINT_OPTIONS) {  
            if (allowed == requested) {  
                return allowed;  
            }  
        }  
    }  
    return DEFAULT_CHART_POINTS;  
}
```

DoctorServlet

Il metodo `doGet()` è solamente uno smistamento a ogni schermata di lettura selezionata dal medico, le quali avranno le loro sottofunzioni per le query al database prima di indirizzare alla pagina.

AdministrationServlet

Nella servlet dell'amministratore è gestita come per il medico, dove si ha il metodo `doGet()` che indirizza verso la richiesta di lettura richiesta. Differente è per la richiesta di registrazione di un nuovo utente, la quale viene effettuata tramite il `doPost()`.

```
private void registerUser(HttpServletRequest request,
                        HttpServletResponse response) throws IOException {

    HttpSession session = request.getSession(true);
    String firstName = request.getParameter("nome");
    String lastName = request.getParameter("cognome");
    String email = request.getParameter("mail");
    String roleParam = request.getParameter("ruolo");

    if (isBlank(firstName) || isBlank(lastName)
        || isBlank(email) || isBlank(roleParam)) {
        Flash.error(request, "Compila tutti i campi");
        redirect(request, response, Pages.ADMIN_SERVLET);
        return;
    }
    Role role = parseRole(roleParam);
    if (role == null) {
        Flash.error(request, "Ruolo non valido.");
        redirect(request, response, Pages.ADMIN_SERVLET);
        return;
    }

    User newUser = new User(email.trim(), firstName.trim(),
                            lastName.trim(), role);
    UserDAO userDAO = daoFactory.getUserDAO();
    String temporaryPassword = userDAO.register(newUser);

    if (temporaryPassword != null) {
        Flash.success(request, "Utente registrato: "+email.trim()+".");
        session.setAttribute("passwordGenerata", temporaryPassword);
    } else {
        Flash.error(request, "Utente non registrato");
    }
    redirect(request, response, Pages.ADMIN_SERVLET);
}
```

Per la creazione di un nuovo utente si effettuano i diversi controlli dell'inserimento, per poi registrare l'utente nel database.

Servlet di Servizio

Tutti i servizi in cui l'utente chiede una visualizzazione o registra un dato sono gestiti da servlet dedicate, che seguono lo stesso schema di funzionamento, ove il metodo `doGet()` serve per reindirizzare per la lettura e il metodo `doPost()` per registrare un nuovo dato.

Servlet e indirizzo	Metodo	Descrizione
MeasurementServlet /patient/measurement	GET POST	Registra le misurazioni della glicemia. Il valore passa due controlli distinti: la plausibilità e la classificazione clinica. Il momento del pasto è obbligatorio, perché senza di esso non si saprebbe quale soglia applicare. L'avviso al medico si scrive solo per i valori fuori soglia, e solo dopo che la registrazione è riuscita: altrimenti il medico riceverebbe una segnalazione che rimanda a una misurazione inesistente.
IntakeServlet /patient/intake	GET POST	Registra le assunzioni di farmaco. Legge la terapia e calcola lo scostamento, se è entro il 10%: nessuna segnalazione; oltre: avviso WARNING ; oltre il doppio o sotto la metà: CRITICAL .
ProblemServlet /patient/problem	GET POST	Segnalazione di sintomi. Il problema deve riferirsi a una terapia oppure a una patologia. Qui l'avviso al medico è sempre dovuto.
NotificationServlet /patient/notification	GET	Elenca le notifiche del paziente, aperte e già prese in carico, con lo stato accanto a ognuna.
PatientMessageServlet /patient/message	GET POST	Conversazione con il medico di riferimento.
MedicalRecordServlet /doctor/record	POST GET reindirizza	Aggiorna i nove fattori di rischio e l'elenco delle patologie. Contiene il controllo <code>findAllPathologies</code> restituisce una lista vuota in caso di errore, la pagina disegna zero caselle, e "nessuna casella spuntata" arriva al server identico a "elenco mai mostrato". Senza il campo nascosto <code>elenco_patologie</code> verrebbe letto come se il paziente non ha patologie.
TherapyServlet /doctor/therapy	POST GET reindirizza	Prescrive, modifica e disattiva le terapie.

Servlet e indirizzo	Metodo	Descrizione
AlertServlet /doctor/alert	GET POST	La GET elenca le segnalazioni aperte, la POST le chiude: due verbi diversi perché leggere non cambia nulla ed e' ripetibile, mentre chiudere cambia lo stato e dopo il redirect non deve essere ripetibile.
DoctorMessageServlet /doctor/message	GET POST	Due schermate: l'elenco dei pazienti fra cui scegliere e la singola conversazione.

Pages

Nella classe `Pages` sono contenuti tutti i percorsi, evitando così problemi di refuso e aumenta la comprensione del codice:

```
...

public static final String PATIENT_HOME = "/Area/Patient/Home.jsp";

public static final String PATIENT_PATHOLOGIES =
    "/Area/Patient/Patologie.jsp";

...
```

Inoltre contiene anche parti componibili degli indirizzi:

```
public static final String DOCTOR_PATIENTS = DOCTOR_SERVLET +
    "?vista=pazienti";
```

Html

```
public static String esc(Object value) {
    if (value == null) {
        return "";
    }
    return StringEscapeUtils.escapeHtml4(value.toString());
}
```

Questa classe difende dal XSS, ovvero quando avviene una iniezione nell'html con uno script leggibile dal browser. Tramite `escapeHtml4` i caratteri speciali vengono sostituiti con le corrispondenti entità.

Flash e FlashMessage

Dopo una chiamata POST viene sempre effettuato un redirect, ricaricando la pagina si rischierebbe di reinviare la richiesta di scrittura, inoltre gli attributi in request verrebbero cancellati. L'unico contenitore che sopravvive è il file di sessione, si deposita quindi il messaggio nella sessione

```
private static final String SESSION_KEY = "flashMessages";

public static void add(HttpServletRequest request,
                      FlashMessage.Level level, String text) {
    if (text == null || text.isBlank()) {
        return;
    }

    HttpSession session = request.getSession(true);

    @SuppressWarnings("unchecked")
    List<FlashMessage> messages =
        (List<FlashMessage>) session.getAttribute(SESSION_KEY);

    if (messages == null) {
        messages = new ArrayList<>();
    }
    messages.add(new FlashMessage(level, text));
    session.setAttribute(SESSION_KEY, messages);
}
```

Sotto la chiave "flashMessages" si mette una lista di FlashMessage, oggetti che portano il testo e il livello (SUCCESS, INFO, WARNING, ERROR). È una lista perché una stessa schermata può mostrarne più di uno, per esempio la registrazione glicemica produce sia la conferma della registrazione sia l'avvertimento sul valore.

Il messaggio nel file di sessione resterebbe per tutta la durata della sessione, ricomparendo su ogni pagina successiva. Indi per cui bisogna toglierlo quando viene letto.

```

public static List<FlashMessage> consume(HttpServletRequest request) {
    HttpSession session = request.getSession(false);
    if (session == null) {
        return Collections.emptyList();
    }

    List<FlashMessage> messages =
        (List<FlashMessage>)session.getAttribute(SESSION_KEY);

    if (messages == null || messages.isEmpty()) {
        return Collections.emptyList();
    }
    session.removeAttribute(SESSION_KEY);

    return messages;
}

```

Da `FlashMessage` la pagina ricava `getCssClass()` e `getIcon()`, per scegliere colore e icona del banner.

AppContextListener

Tomcat lo esegue all'avvio del contesto in modo tale che scelga l'implementazione da usare e che avvii il thread demone (con annesso delay di avvio per il corretto avvio di Tomcat).

```

@Override
public void contextInitialized(ServletContextEvent sce) {
    connectionManager = new MySqlConnectionManager();
    DAOFactory daoFactory = new JdbcDAOFactory(connectionManager);
    ServletContext context = sce.getServletContext();
    context.setAttribute(DAO_FACTORY_KEY, daoFactory);
    startAdherenceMonitor(daoFactory);
}

private void startAdherenceMonitor(DAOFactory daoFactory) {
    scheduler = Executors.newSingleThreadScheduledExecutor(runnable -> {
        Thread thread = new Thread(runnable, "adherence-monitor");
        thread.setDaemon(true);
        return thread;
    });

    scheduler.scheduleAtFixedRate(
        new AdherenceMonitor(daoFactory),
        FIRST_RUN_DELAY_MINUTES,
        TimeUnit.HOURS.toMinutes(PERIOD_HOURS),
        TimeUnit.MINUTES);
}

```

Allo spegnimento bisogna fermare anche il thread, perchè esso rimarrebbe e mantiene un riferimento al contesto, avviando così Tomcat e non liberando memoria.

Usando `shutdown()` e `shutdownNow()` si interrompe il meccanismo, dove inizialmente viene usato `shutdown`, ma se dopo 10s è ancora attivo allora viene invocato `shutdownNow` che interrompe immediatamente. In caso di errore bisogna ripristinare i flag di interruzione.

```
@Override
    public void contextDestroyed(ServletContextEvent sce) {
        if (scheduler == null) {
            return;
        }

        scheduler.shutdown();
        try {
            if (!scheduler.awaitTermination(SHUTDOWN_TIMEOUT_SECONDS,
                TimeUnit.SECONDS)) {
                scheduler.shutdownNow();
            }
        } catch (InterruptedException e) {
            scheduler.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}
```

8.4 - Package service

AdherenceMonitor

Il controllo periodico se i pazienti continuano a seguire le proprie terapie attive avviene tramite questa classe. Questo controllo viene eseguito a intervalli regolari da un thread demone avviato dal `AppContextListener`. Per creare il thread la classe deve implementare `Runnable`, la quale impone il metodo `run()`.

```
@Override
public void run() {
    try {
        check();
    } catch (Throwable t) {
        // cancella il compito periodico di fronte a QUALSIASI Throwable
    }
}
```

Il controllo della soglia si calcola sottraendo `DAYS_WITHOUT_INTAKE` dall'istante corrente. Poi si effettua la ricerca nel database in base alla data dell'ultima assunzione.

```
int check() {
    LocalDateTime threshold =
        LocalDateTime.now().minusDays(DAYS_WITHOUT_INTAKE);
    List<MissedIntake> missed =
        daoFactory.getTherapyDAO().findMissedIntakes(threshold);
    if (missed.isEmpty()) {
        return 0;
    }
    AlertDAO alertDAO = daoFactory.getAlertDAO();
    int created = 0;
    for (MissedIntake intake : missed) {
        String message = buildMessage(intake);
        if (alertDAO.existsOpen(intake.getPatientId(),
            Alert.TYPE_ADHERENCE, message)) {
            continue;
        }
        boolean saved = alertDAO.create(intake.getPatientId(),
            Alert.RECIPIENT_DOCTOR, Alert.TYPE_ADHERENCE,
            AlertSeverity.CRITICAL, message);
        if (saved) {
            created++;
        }
    }
    return created;
}
```

Il controllo viene eseguito ogni giorno, se un paziente continua a non seguire la terapia si rischia che il medico sia sommerso da alert, quindi prima di creare l'alert si verifica se ce ne sia già uno uguale aperto tramite `existsOpen`.

Quando il medico lo prende in carico l'avviso e il paziente continua a non seguire la terapia, l'alert viene ricreato.

Bisogna distinguere però il caso se il paziente non ha mai iniziato la terapia, per questo alla formattazione del messaggio dell'avviso al medico si specifica se il paziente non ha mai iniziato o non la sta più seguendo.

```
private String buildMessage(MissedIntake intake) {
    String drug = (intake.getDrugName() != null) ?
        intake.getDrugName() : "farmaco prescritto";
    String reference = " (terapia n. " + intake.getTherapyId() + ")";

    if (intake.isNeverTaken()) {
        return "Nessuna assunzione di " + drug + reference
            + " mai registrata: la terapia non iniziata.";
    }

    return "Nessuna assunzione di " + drug + reference
        + " registrata da oltre " + DAYS_WITHOUT_INTAKE + " giorni.";
}
```

9 - Test

9.1 - Obiettivo e metodologia

Questa sezione riporta 49 test eseguiti sul progetto **DiabetesPortal** per verificarne il corretto funzionamento, sia dal punto di vista dell'**integrità del codice** (il progetto compila, si avvia, non genera eccezioni, rispetta le pratiche di sicurezza dichiarate) sia dal punto di vista dell'**utente finale** (paziente, medico, amministratore), simulando l'uso reale dell'applicazione attraverso il browser/HTTP

Il progetto non contiene una cartella `src/test` nè alcun test automatico preesistente: tutti i test descritti in questo capitolo sono stati progettati ed eseguiti da zero per questa verifica.

Ambiente di Test

Per ottenere risultati reali (non simulati o dedotti dalla sola lettura del codice) e allo stesso tempo non toccare in alcun modo l'ambiente di sviluppo esistente sulla macchina è stato allestito un ambiente di test completamente isolato:

Componente	Dettaglio
Build applicativa	WAR compilato dal sorgente corrente con <code>mvn clean package</code>
Server applicativo	Istanza dedicata di Apache Tomcat 10.1.55, separata da qualunque altro Tomcat presente sulla macchina, su una porta HTTP privata
Database	Istanza dedicata di Mysql, con uno schema <code>DBDiabetes</code> creato appositamente e popolato solo con i dati generati dai test stessi
Schema del database	Ricostruito leggendo tutte le query SQL presenti nelle classi <code>Jdbc*DAO</code>
Utente amministratore iniziale	Creato manualmente con una password nota (hash BCrypt calcolato con lo stesso algoritmo dell'applicazione), perchè il portale non offre un'auto registrazione: senza questo passaggio non sarebbe stato possibile accedere nemmeno una prima volta
Variabile segnaposto \$WORK	Variabile segnaposto che viene usata negli esempi di comando per indicare una cartella vuota a scelta dove si vuole che ci siano i file temporanei, database di prova, configurazione di Tomcat, script in python

Come sono stati Eseguiti i Test

Sono state usate tre tecniche diverse, scelte in base a cosa ciascun test doveva verificare:

1. **Test da riga di comando:** esecuzione reale di `mvn clean compile` e `mvn clean package` con relativa ispezione dell'output
2. **Test HTTP end-to-end:** uno script, python, impersona in sessioni HTTP separate un amministratore, due medici e un paziente, inviando le stesse richieste POST/GET che invierebbe un browser (login, registrazione, compilazione dei moduli, invio misurazioni, prescrizioni, messaggi ecc ecc), verificando i codici di stato, redirect, contenuto delle pagine.
3. **Test diretti sul livello di servizio - harness Java:** per due componenti non raggiungibili da una normale richiesta HTTP, il controllo periodico `AdherenceMonitor` (pensato per girare da solo ogni 24 ore) e un vincolo interno del DAO delle segnalazioni, è stata scritta una classe Java che li invoca direttamente, nello stesso pacchetto, usando la stessa configurazione di database dell'applicazione

Per completare, alcuni test sono di **revisione statica del codice sorgente** (lettura di classi alla ricerca di un pattern preciso)

usata quando il comportamento da verificare non è osservabile dall'esterno.

Riproducibilità: Script e Comandi usati

Tutti i comandi e gli script elencati sono forniti così come sono stati eseguiti per permettere di ripetere l'intera sessione di test in autonomia.

I quattro script in Python e la classe Java sono disponibili nella cartella

`Documentazione/script-test` del progetto insieme allo schema del database `schema.sql`.

Allestimento dell'Ambiente

1. Scegliere una password per l'amministratore: Esempio `MiaPasswordAdimin1!`
2. Creare la cartella di lavoro per il database e il server di test

```
#Puntatore della cartella WORK ad una cartella reale, esempio
WORK=~/.dp-test
mkdir -p "$WORK/mysql/data"
```

3. Mysql

```
# Database dedicato e isolato, non quello di sistema
mysqld --no-defaults --initialize-insecure --datadir="$WORK/mysql/data"
```

```
/usr/sbin/mysqld --no-defaults --datadir="$WORK/mysql/data" \  
--socket=/tmp/dp_mysql.sock --port=33061 --bind-address=127.0.0.1 \  
--pid-file=/tmp/dp_mysql.pid --secure-file-priv=""&
```

4. Creare il database, lo schema e l'amministratore: ci si deve spostare nella cartella dove si ha salvato `schema.sql`

```
mysql -u root --socket=/tmp/dp_mysql.sock -e "CREATE DATABASE DBDiabetes  
CHARACTER SET utf8mb4;"
```

```
mysql -u root --socket=/tmp/dp_mysql.sock < schema.sql
```

5. Generare l'hash della password dell'admin scelta al punto 1: stessa cartella di prima

1. `find ~/.m2 -name "jbcrypt-0.4.jar"`
2. Copia il percorso stampato e lo si metta al posto di JARPATH nei due comandi seguenti:

```
cat > GenHash.java <<'EOF'  
import org.mindrot.jbcrypt.BCrypt;  
public class GenHash {  
    public static void main(String[] args) {  
        System.out.println(BCrypt.hashpw("MiaPasswordAdmin1!",  
BCrypt.gensalt(12)));  
    }  
}  
EOF
```

```
javac -cp "JARPATH" GenHash.java  
java -cp ".:JARPATH" GenHash
```

Il secondo comando stampa una riga molto lunga è l'HASH, serve per il passo successivo.

```
mysql -u root --socket=/tmp/dp_mysql.sock -e "USE DBDiabetes; INSERT INTO  
UTENTE  
(mail, nome, cognome, password, nuovo, ruolo)  
VALUES  
( 'admin@diabetesportal.test', 'Admin', 'Sistema', '\$2a\$12\$6xlq2Lc0u64Ctafy  
J3KtXec9Cg2bw80/WTaarWmjEGYLEy.VstpL2', 0, 2);"
```

6. Inserire l'amministratore nel database

```
# Amministratore iniziale (hash BCrypt generato con la stessa libreria del progetto)
mysql -u root --socket=/tmp/dp_mysql.sock -e "USE DBDiabetes; INSERT INTO UTENTE (mail, nome, cognome, password, nuovo, ruolo) VALUES ('admin@diabetesportal.test', 'Admin', 'Sistema', 'HASH', 0, 2);"
```

```
#Per verificare se tutto procede come deve
mysql -u root --socket=/tmp/dp_mysql.sock -e "USE DBDiabetes; SELECT mail, password FROM UTENTE;"
```

7. Configurazione del progetto e compilazione: ci si posta nella cartella radice del progetto

```
cat > src/main/resources/db.properties <<'EOF'
db.url=jdbc:mysql://127.0.0.1:33061/DBDiabetes?
useSSL=false&serverTimezone=Europe/Rome
db.user=root
db.password=
EOF
```

```
mvn -o clean package
```

8. Preparare e avviare Tomcat

```
# db.properties del progetto punta temporaneamente al DB di test, poi:
mkdir -p $WORK/tomcat/{conf,logs,temp,webapps,work}

cp -r /usr/share/tomcat10/etc/* $WORK/tomcat/conf/

sed -i 's/port="8080"/port="18080"/' $WORK/tomcat/conf/server.xml

cp target/DiabetesPortal-1.0-SNAPSHOT.war $WORK/tomcat/webapps/DiabetesPortal.war
CATALINA_HOME=/usr/share/tomcat10 CATALINA_BASE=$WORK/tomcat /usr/share/tomcat10/bin/catalina.sh start
```

9. Verifica finale ed accesso

```
curl -s -o /dev/null -w "%{http_code}\n"
http://localhost:18080/DiabetesPortal/
```

Deve stampare 200. Aprire il browser su <http://localhost:8080/DiabetesPortal/> ed entrare con:

- Email: admin@diabetesportal.test
- Password: MiaPasswordAdmin1! (quella scelta al Passo 1)

Script di Test HTTP

Script	Cosa esegue	Test Prodotti
run_tests.py	Login amministratore, registrazione medico/paziente/secondo medico, tentativi di registrazione non validi, accesso non autenticato, credenziali errate, SQL injection sul login	A01-A09, AD01-AD07, S04
run_tests2.py	Primo accesso di medico e paziente: cambio password obbligatorio con casi di rifiuto, compilazione dati anagrafici, cartella clinica, scelta del medico e delle patologie	DR01-DR06, PZ01-PZ07
run_tests3.py	Misurazioni glicemiche (normali, basse, alte, impossibili), prescrizione/modifica terapia, assunzioni farmaco incluso il caso di sovra-dosaggio	M01-M07, AL01, T01-T07
run_tests4.py	Segnalazioni problemi, messaggistica paziente-medico, risoluzione avvisi, autorizzazione incrociata dei medici, XSS, errori	P01-P06, MS01-MS03, S01-S03, S05-S08, AL02-AL03

```
cd $WORK/testrun
python3 run_tests.py
python3 run_tests2.py
python3 run_tests3.py
# retrodatazione terapia + AdherenceHarness, vedi 1.3.3
python3 run_tests4.py
```

Ogni script legge lo stato lasciato dal precedente (credenziali generate, id creati) da un file `state.json` nella stessa cartella, e vi aggiunge i risultati in `results_partial*.json` : vanno quindi eseguiti in sequenza, nell'ordine indicato, dalla stessa directory di lavoro.

Test sul Livello di Servizio

`AdherenceMonitor` agisce sulle terapie priva di assunzioni da almeno 3 giorni: perchè il test AM01 trovi qualcosa da generare, una terapia viene prima retrodatata di 10 giorni, poi il servizio viene invocato due volte di seguito (la seconda per verificare che non generi un avviso duplicato) tramite una piccola classe eseguita nello stesso pacchetto Java dell'applicazione.

```
# Retrodatazione (id_terapia=1 nel dataset di test)
mysql -u root --socket=/tmp/dp_mysql.sock -e "USE DBDiabetes;
UPDATE TERAPIA SET data_inizio = DATE_SUB(CURDATE(), INTERVAL 10 DAY)
WHERE id=1;
UPDATE ASSUNZIONE SET data_ora = DATE_SUB(NOW(), INTERVAL 10 DAY) WHERE
id_terapia=1;"
```

```
# Compilazione ed esecuzione dell'harness (AdherenceHarness.java)
mvn -o dependency:build-classpath -Dmdep.outputFile=/tmp/cp.txt
CP="$(cat /tmp/cp.txt):target/classes:src/main/resources"
javac -cp "$CP" -d harness-out AdherenceHarness.java
java -cp "$CP:harness-out"
com.mycompany.diabetesportal.service.AdherenceHarness
```

Lo stesso programma esegue anche i test P03 e P03b, invocando direttamente `PatientDA0.recordProblem()` con entrambi i riferimenti (terapia e patologia) valorizzati e poi con nessuno dei due, per verificare il vincolo di esclusività che a livello HTTP non è possibile forzare (il sistema invia sempre e solo uno dei due per come è costruito).

Attenzione se si esegue harness

Gli identificativi (3 per il paziente, 2 per il medico, 1 per la terapia, 2 per il conteggio degli avvisi del medico) sono collegati nel file e corrispondono all'ordine di creazione degli account usati dagli script qua sopra. Un ordine diverso richiede di aggiornare questi valori verificabili con un semplice `SELECT id FROM UTENTE WHERE email=...`

9.2 - Legenda

Ogni riga della tabella riporta:

- **ID** del test
- **Obiettivo**
- **Come è stato eseguito**
- **Evidenza raccolta**

L'ultima colonna riporta l'esito: **SUPERATO** se il comportamento osservato corrisponde a quello atteso, **NON SUPERATO** in caso contrario.

I test sono raggruppati per area funzionale.

9.3 - Tabella dei Test Eseguiti

Integrità del Codice

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
B01	Il progetto compila senza errori con <code>mvn clean compile</code>	<code>mvn -o clean compile</code>	57 file sorgente compilati, 0 errori	SUPERATO
B02	Il progetto produce un pacchetto WAR completo con <code>mvn clean package</code>	<code>mvn -o clean package</code>	DiabetesPortal-1.0-SNAPSHOT.war generato, 5.166.224 byte	SUPERATO
B03	Il WAR si avvia su un'istanza reale di Apache Tomcat 10.1.55 senza eccezioni nei log	Deploy su istanza Tomcat privata dedicata al test e ispezione di catalina.out	nessuna riga SEVERE/Exception /Error nei log di avvio	SUPERATO
B04	Nessuna eccezione non gestita compare nei log del server durante l'intera sessione di test	Ispezione di catalina.out al termine della sessione di test	0 occorrenze di SEVERE/Exception /Error	SUPERATO
B05	Tutte le query nei DAO JDBC	Lettura sistematica del codice sorgente di tutti i <code>Jdbc*DAO.java</code>	verificato manualmente su JdbcUserDAO, JdbcPatientDAO, JdbcDoctorDAO, JdbcTherapyDAO, JdbcAlertDAO, JdbcMessageDAO	SUPERATO
B06	persistence.xml e classi JakartaRest-Configuration/ JakartaEE11-Resource sono presenti ma inutilizzati: nessuna classe li richiama, la persistenza reale e' tutta JDBC manuale	Lettura di persistence.xml e delle classi di scaffolding; ricerca di riferimenti a EntityManager ed endpoint REST nel resto del codice e nelle JSP	nessun uso di jakarta.persistence.* ne' di endpoint REST nel codice applicativo: scaffolding residuo dell'archetipo NetBeans, innocuo ma da rimuovere per pulizia	SUPERATO

Autenticazione e Sessione

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
A01	Login amministratore con credenziali valide	POST /login	status=302 location= /DiabetesPortal /administration	SUPERATO
A02	Login con password errata viene rifiutato con messaggio	POST /login	-	SUPERATO
A03	Login con email inesistente restituisce lo stesso messaggio	POST /login	verifica che i due messaggi siano testualmente identici	SUPERATO
A08	Accesso a /administration senza login viene rifiutato reindirizzato	GET /administration (no session)	status=302 loc=/DiabetesPortal /index.jsp	SUPERATO
A09	Cookie JSESSIONID impostato con attributo HttpOnly	Set-Cookie header su login	JSESSIONID= 82D78A545997CDF 41D21AE009F960394; Path= /DiabetesPortal; HttpOnly	SUPERATO

Area Amministrazione

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
AD01	Registrazione nuovo medico con dati validi	POST /administration	password temporanea: presente	SUPERATO
AD02	Registrazione nuovo paziente con dati validi	POST /administration	password temporanea: presente	SUPERATO
AD03	Registrazione con email duplicata viene rifiutata	POST /administration	-	SUPERATO
AD04	Registrazione con campi mancanti viene rifiutata	POST /administration	-	SUPERATO

Primo Accesso e Area Medico

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
DR01	Primo login con password temporanea forza il cambio password	POST /login	redirect -> /DiabetesPortal /Area/ ChangePassword.jsp	SUPERATO
DR02	Cambio password rifiutato se troppo corta (<8 caratteri)	POST /login (change)	-	SUPERATO
DR04	Cambio password valido reindirizza all'area applicativa	POST /login (change)	redirect -> /DiabetesPortal /doctor	SUPERATO
DR05	Compilazione dati anagrafici medico va a buon fine	POST /doctor	-	SUPERATO

Primo Accesso e Area Paziente

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
PZ01	Primo login con password temporanea forza il cambio password	POST /login	-	SUPERATO
PZ02	Cambio password valido reindirizza all'area applicativa	POST /login (change)	-	SUPERATO
PZ04	Compilazione dati anagrafici e cartella clinica va a buon fine	POST /patient	atteso redirect alla scelta medico/patologie	SUPERATO
PZ06	Scelta medico di riferimento e patologie va a buon fine	POST /patient	doc_id=2 path_ids=['1','2']	SUPERATO

Misurazioni Glicemiche

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
M01	Registrazione glicemia normale (95 mg/dL a digiuno) va a buon fine, nessun alert critico	POST /patient/measurement	-	SUPERATO
M02	Registrazione glicemia molto bassa (40 mg/dL) genera avviso critico per il medico	POST /patient/measurement	-	SUPERATO
M03	Registrazione glicemia alta dopo pasto (210 mg/dL) genera avviso per il medico	POST /patient/measurement	-	SUPERATO
M04	Valore impossibile (900 mg/dL) viene rifiutato	POST /patient/measurement	-	SUPERATO

Terapie e Assunzioni farmaci

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
T01	Il medico prescrive una nuova terapia al proprio paziente	POST /doctor/therapy azione=nuova	id_terapia creato=1	SUPERATO
T02	Prescrizione con quantita' <= 0 viene rifiutata	POST /doctor/therapy quantità=0	terapie in DB dopo il tentativo=1	SUPERATO
T03	Il medico modifica una terapia attiva del proprio paziente	POST /doctor/therapy azione=modifica	quantita' in DB dopo modifica=850	SUPERATO
T05	Assunzione di dose doppia rispetto al prescritto genera un avviso critico per il medico	POST /doctor/therapy overdose	avvisi ASSUNZIONE dopo overdose=1	SUPERATO

Segnalazione Problemi/Sintomi

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
P01	Segnalazione problema legata a una terapia va a buon fine	POST /patient/problem	-	SUPERATO
P02	Segnalazione problema legata a una patologia va a buon fine	POST /patient/problem	-	SUPERATO
P03	recordProblem rifiuta una segnalazione con sia id_terapia sia id_patologia valorizzati (violazione del vincolo XOR)	Invocazione diretta di PatientDAO. recordProblem() con entrambi i riferimenti (harness Java)	metodo ha restituito false; conteggio righe PROBLEMA invariato (2) dopo la chiamata	SUPERATO
P04	Segnalazione senza alcun sintomo selezionato viene rifiutata	POST /patient/problem sintomi assenti	-	SUPERATO

Segnalazioni e avvisi Alert

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
AL02	Il medico in carico puo' risolvere un avviso del proprio paziente	POST /doctor/alert	risolto=1	SUPERATO
AL03	Un medico non in carico NON può risolvere l'avviso di un paziente altrui	POST /doctor/alert (medico non autorizzato)	risolto dopo tentativo non autorizzato=0 (atteso 0)	SUPERATO

Messaggistica Paziente-Medico

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
MS01	Il paziente invia un messaggio al proprio medico di riferimento	POST /patient/message	-	SUPERATO
MS03	Il medico risponde nella conversazione	POST /doctor/message	messaggi in DB=2	SUPERATO

Controllo Aderenza alla Terapia

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
AM01	Il controllo periodico rileva una terapia attiva senza assunzioni da oltre 7 giorni e crea un avviso CRITICAL per il medico	Invocazione diretta di AdherenceMonitor.check() (harness Java, stessa JVM/DB)	alert aperti: 4 - > 5 dopo il giro; nuovo alert ADERENZA severita CRITICAL creato in ALERT	SUPERATO
AM02	Una seconda esecuzione immediata del controllo NON crea un avviso duplicato per la stessa terapia disattesa	Invocazione diretta di AdherenceMonitor.check() due volte di seguito	0 nuovi alert creati al secondo giro	SUPERATO

Sicurezza Applicativa

ID	OBIETTIVO	COME È STATO ESEGUITO	EVIDENZA RACCOLTA	ESITO
S01	Un paziente non può accedere all'area medico	GET /doctor (sessione paziente)	status=302 loc= /DiabetesPortal /index	SUPERATO
S02	Un paziente non può accedere all'area amministrazione	GET /administration (sessione paziente)	status=302 loc =/DiabetesPortal/ ind	SUPERATO
S03	Un medico non in carico non può leggere la conversazione di un paziente altrui	GET /doctor/message? paziente=3	-	SUPERATO
S04	Tentativo di SQL injection nel form di login non produce accesso	POST /login (payload SQLi)	status=200 (200=resta sul login, ness	SUPERATO
S05	Un medico non in carico non può vedere la cartella clinica di un paziente altrui	GET /doctor? paziente=3 &sezione=cartella	-	SUPERATO
S06	Testo HTML/script inserito dal paziente in un messaggio viene mostrato al medico correttamente escapato (no < script > eseguibile)	GET /doctor/message? paziente=3	-	SUPERATO
S07	Una risorsa inesistente restituisce la pagina di errore generica (404)	GET /percorso- inesistente	status=404	SUPERATO
S08	Un id paziente non numerico non genera un errore 500 (gestito in modo controllato)	GET /doctor? paziente=abc	status=200	SUPERATO